

Semantic-Driven Context Pruning for Arabic RAG Systems: Toward Memory-Efficient vLLM-Based Deployment

Akram Taha^{1,2*}

^{1*}College of Computer Engineering, University of Technology - Iraq,
Baghdad, Iraq.

²Center for Artificial Intelligence Technology (CAIT), Faculty of
Information Science and Technology (FTSM), Universiti Kebangsaan
Malaysia (UKM), Bangi, Selangor, Malaysia.

Corresponding author(s). E-mail(s): akramtaha30@gmail.com;

Abstract

Retrieval-Augmented Generation (RAG) for Arabic faces a memory bottleneck: Arabic’s rich morphology yields more tokens than English text, inflating the Key-Value (KV) cache engines like vLLM must maintain per request. We introduce a Lightweight Semantic Pruning Middleware (LSPM) that scores retrieved passages at the sentence level with a cross-encoder, keeping only the most relevant subset at a fixed or dynamic ratio. We report five preliminary findings. First, across 30 verified Arabic/English sentence pairs, two tokenizers confirm a token ratio of 1.76x/1.69x (bootstrap $p < 0.0001$ vs. parity). Second, a fidelity pilot ($r=0.3-0.7$) shows pruned answers match unpruned ones (ROUGE-L 0.89-0.93, BERTScore-F1 0.96-0.97). Third, on 140 ARCD questions (980 generations), LSPM beats naive truncation on F1 at the most aggressive ratio (+0.114, adjusted $p = 0.005$) and is equivalent to the unpruned answer ($p = 0.038, 0.003$); naive truncation is significantly worse than unpruned at two ratios (adjusted $p = 0.0004, 0.015$). Fourth, a GPU benchmark shows LSPM lowers KV-cache occupancy at every ratio at $c = 1$, with 8.5-11.9% lower throughput and near-unchanged token rate, exploratory only since it reverses at higher concurrency. Fifth, two blind evaluators rated 90 answers for correctness and faithfulness (kappa 0.79, 0.94): LSPM scored at least as high as raw and higher than naive truncation, consistent with the F1 result above. We release the implementation, raw data, and a live demo.

Keywords: retrieval-augmented generation, Arabic natural language processing, prompt compression, vLLM, large language models

ORCID (A. Taha): <https://orcid.org/0009-0002-4020-8060>

1 Introduction

Large Language Models (LLMs), built on the Transformer architecture [1] and exemplified by models such as GPT-3 [2] and its successors, have become the default reasoning engine behind a rapidly growing class of production systems, and Retrieval-Augmented Generation (RAG) has become the default architecture for grounding those systems in external, up-to-date, or proprietary knowledge rather than relying solely on parametric memory [3]. In a canonical RAG pipeline, a retriever selects the top-k passages most relevant to a user query from a vector index, and those passages are concatenated into the LLM’s context window alongside the query itself before generation. The approach is simple, effective, and now underlies everything from enterprise search assistants to customer-support chatbots. It is also, in practice, expensive: every additional retrieved token that enters the context window has to be attended over during the prefill stage of inference, and, critically for serving systems built on modern memory-efficient engines, every token also occupies a slot in the Key-Value (KV) cache that must be held in GPU memory for the duration of the request [4].

vLLM has emerged as one of the most widely adopted open-source inference engines precisely because it addresses this memory pressure directly. Its PagedAttention algorithm borrows the paging abstraction from operating-systems virtual memory to store the KV cache in fixed-size, non-contiguous blocks, eliminating the internal fragmentation that plagued earlier contiguous-allocation serving stacks and allowing far higher batching throughput under concurrent load [4]. PagedAttention and the broader family of KV-cache management techniques it inspired (attention-sink-based streaming [5], heavy-hitter-oracle eviction [6], and iteration-level continuous batching [7]) have collectively become close to an industry norm for LLM serving. But all of these techniques manage the consequences of a large KV cache; none of them reduce the number of tokens that create it in the first place. That is the gap this paper addresses, specifically for Arabic.

Arabic is a morphologically rich, templatic language: a single trilateral root can surface as dozens of distinct word forms through internal vowel changes, affixation, and clitic attachment, and standard subword tokenizers (byte-pair encoding [8] and its relatives, which are near-universally trained on English-dominated corpora) fragment Arabic text far more aggressively than they fragment English text of equivalent semantic content. This "tokenization disparity" is not a minor curiosity; it means that for a fixed context budget, an Arabic RAG system can retrieve meaningfully less semantic content than an English one, and that for a fixed amount of retrieved semantic content, an Arabic RAG system imposes a meaningfully larger KV-cache footprint on the serving engine. Under concurrent multi-user traffic, the normal operating condition

for any production deployment, this directly translates into higher memory pressure, lower achievable batch sizes, and, ultimately, higher latency and lower throughput.

The natural response inside the NLP community has been prompt and context compression: methods such as LLMingua [9], Selective Context [10], and RECOMP [11] all attempt to shrink the text that enters an LLM’s context window while preserving the information the model needs to answer correctly. These methods are general-purpose and largely English-centric in their evaluation, and none of them is designed with the Arabic tokenization disparity, or with vLLM’s specific KV-cache mechanics, as a first-class design constraint. Our contribution sits at the intersection of these two threads.

We propose a **Lightweight Semantic Pruning Middleware (LSPM)**: a thin layer inserted between the retriever and the generator in an Arabic RAG pipeline that (i) splits every retrieved passage into sentences, (ii) scores each sentence’s relevance to the query with a cross-encoder reranking model (the same class of model used for passage reranking in modern retrieval pipelines [12], [13]), and (iii) reconstructs a compact, order-preserving context containing only the highest-scoring sentences, under a compression ratio that can either be fixed by the operator or computed dynamically from vLLM’s live `/metrics` endpoint so that pruning tightens automatically as GPU KV-cache utilization rises.

A note on scope. This paper is primarily an architecture-and-preliminary-validation study, not a full systems benchmark paper, though it now includes one real, if narrowly scoped, systems measurement. We report an analytical (not measured) projection of KV-cache savings in Section 5.5, and a measured throughput/TTFT/KV-cache benchmark against a self-hosted vLLM server in Section 5.8. Only the single lowest-concurrency cell ($c = 1$) supports the projection’s direction on KV-cache occupancy, and even there it comes with a measurable 8.5-11.9% lower request throughput alongside a near-unchanged ($<0.5\%$) completion-token rate, a tradeoff we report plainly rather than describe as free; already at the next concurrency level tested ($c = 10$) the KV-cache comparison is mixed, and by $c \geq 25$ it reverses outright, a pattern we attribute to an identified, specified methodological confound (Section 4.6, 5.8, 7, 8) rather than reporting as a validated claim. Readers looking for a fully validated, high-concurrency systems benchmark will not find one in this submission; readers looking for a preliminary compression architecture with honestly reported evidence, including a baseline comparison re-tested at ARCD scale with ground-truth scoring, a single-cell exploratory real-GPU observation, and a correction to our own initial overreading of one result, will find the full picture, uncertainties included, in Sections 4-7.

The specific contributions of this paper are:

1. **A concrete system architecture and open reference implementation** for sentence-level, cross-encoder-driven context pruning for Arabic RAG on vLLM-class serving infrastructure, including a dynamic compression-ratio controller design that couples pruning aggressiveness to real-time GPU KV-cache occupancy; this controller has not itself been evaluated against live vLLM traffic and is presented as an implementation feature, not a validated result.
2. **A quantitative measurement of Arabic-English tokenization disparity** on a 30-pair hand-verified parallel corpus, with a one-sample bootstrap test rejecting

parity on that corpus for both of two tokenizers tested ($p < 0.0001$); the effect is descriptive of this constructed corpus and has not been shown to generalize to a broader sampled distribution of Arabic text.

3. **A ratio-swept semantic-fidelity study** ($r = 0.3, 0.5, 0.7$; $n = 8$ real QA items per ratio; 95% bootstrap confidence intervals) showing that sentence-level pruning preserves downstream answer fidelity across the tested range against a live LLM endpoint, on this pilot’s small corpus.
4. **A real, paired baseline comparison against naive length-matched truncation, run at three scales with a properly powered, Holm-Bonferroni-corrected re-test**, reported with full honesty including a correction to our own initial interpretation of an earlier, underpowered result. On the original small, low-redundancy pilot corpus ($n = 8$), LSPM’s cross-encoder scoring shows no statistically significant fidelity advantage over naive truncation (all bootstrap $p > 0.2$). A first re-test on 40 real Arabic Reading Comprehension Dataset (ARCD) [25] questions against a noisier, answer-guaranteed six-document pool also found the head-to-head comparison statistically unresolved, and a nominal asymmetry we initially read as evidence of LSPM-specific fidelity preservation did not survive Holm-Bonferroni correction. A follow-up, fully powered re-test on 140 ARCD questions (980 live generations, five times the sample size) resolves the question at the most aggressive compression ratio: LSPM significantly outperforms naive truncation head-to-head on token-F1 at $r = 0.3$ (Holm-adjusted $p = 0.005$) and is statistically equivalent to the unpruned answer by a predefined-margin equivalence test at $r = 0.3$ and $r = 0.7$ (TOST, $p = 0.038$, $p = 0.003$), while naive truncation is significantly worse than the unpruned answer at $r = 0.3$ and $r = 0.5$ (adjusted $p = 0.0004$, $p = 0.015$). At $r = 0.5$ and $r = 0.7$ the head-to-head advantage remains directionally positive but does not survive correction, so the comparison is resolved at the most demanding ratio and still open at the more moderate ones (Section 5.4, Section 6).
5. **A smaller, complementary end-to-end retrieval check that removes the answer-guaranteed pool construction** (Section 4.9/5.7): 60 fresh questions retrieved from a real 234-paragraph corpus with a genuine TF-IDF retriever (recall@6 = 86.7%, so retrieval can and does fail), at $r = 0.3$ only. LSPM’s head-to-head advantage over naive truncation persists directionally in this harder setting (EM: +0.100 overall, uncorrected Wilcoxon $p = 0.014$), reported as a single-ratio, exploratory result that complements rather than replaces the fully powered Section 5.4 finding.
6. **An analytical, explicitly-labeled KV-cache savings projection**, derived from Llama-3.1-8B-Instruct’s real, publicly verifiable architecture configuration combined with the real measured context-size reduction, to give readers a concrete sense of the systems-level stakes, and, now, **a measured GPU benchmark that tests its direction** (Section 5.8): against a self-hosted vLLM server, LSPM reduces measured KV-cache occupancy, at every tested compression ratio, at the single, least time-confounded concurrency level ($c = 1$); we report this as an exploratory single-cell observation rather than a validated confirmation, since the comparison

is already mixed at $c = 10$ and reverses by $c \geq 25$ for reasons we diagnose and scope rather than paper over.

7. **A transparent GPU benchmark protocol and the Locust-based harness needed to run it** (Section 4.6/8), used to produce the Section 5.8 result above; the same protocol also surfaced a specific, diagnosed methodological confound at higher concurrency (client-side scoring bottleneck in the load generator; no server restart between method blocks), which we report honestly rather than papering over, together with the exact harness fix a follow-up run needs to resolve it.
8. **A blind, two-rater human evaluation** (Section 4.10/5.9) that addresses the "automatic metrics only" limitation flagged in earlier revisions: 90 real answers (30 ARCD questions x raw/LSPM/naive truncation at $r = 0.3$), rated independently by two bilingual evaluators for correctness (0-2) and faithfulness (0-1) without knowledge of which method produced each answer, with substantial-to-near-perfect inter-rater agreement (Cohen's kappa 0.794 correctness, 0.935 faithfulness). Human judgment agrees directionally with the automatic-metric result in Section 5.4: LSPM scores at least as high as the unpruned answer on both dimensions and higher than naive truncation, though, like the automatic-metric result it corroborates, this specific check covers only $r = 0.3$ and a single 90-item sample.
9. **A preliminary edge-cloud extension with two new rule-based compression-ratio controllers, run on a small validation grid** (Section 4.11/5.10): a KV-cache-only controller and a five-input network-aware controller, both non-learned and both snapping only to fidelity-validated ratios, evaluated across 90 grid cells (5 methods x 2 network profiles x 3 concurrency levels x 3 repeats, application-level network emulation since the rented environment did not grant kernel-level `tc netem` access). The exercise itself found and fixed three real implementation defects (a metric-name mismatch, blocking synchronous calls inside async request handlers, and a controller blind to the active emulation profile), which we disclose alongside the corrected re-run rather than silently. After the fix, the network-aware controller correctly differentiates its ratio by network condition, a result it did not produce beforehand; the same fix also unmasked, rather than removed, a genuine architectural scaling cost from polling vLLM's `/metrics` endpoint once per request under concurrency, which we report as a disclosed limitation with a specific proposed fix (Section 7, 8) rather than as a validated systems win. This extension is small-scale, single-box (edge and cloud roles are logical, not physically distributed), and carries no answer-correctness or faithfulness data; it does not change, and is not presented as changing, any ARCD-scale finding elsewhere in this paper.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 describes the LSPM architecture. Section 4 describes the experimental setup, including the ratio-sweep, baseline-comparison, ARCD-scale ground-truth re-test, end-to-end retrieval check, human evaluation, and the preliminary edge-cloud validation-grid methodology. Section 5 reports results. Section 6 discusses their implications, including the baseline-comparison result and its confirmation at ARCD scale and under real retrieval. Section 7 states limitations candidly. Section 8 outlines future work, and Section 9 concludes.

2 Related Work

2.1 Retrieval-Augmented Generation

RAG was introduced by Lewis et al. as a way to combine a parametric sequence-to-sequence generator with a non-parametric, retrievable memory, jointly fine-tuning both components for knowledge-intensive NLP tasks such as open-domain question answering [3]. The framework proved that grounding generation in retrieved evidence could substantially outperform purely parametric models on tasks that require access to specific, long-tail, or time-sensitive facts, without requiring the model to memorize that information in its weights. Subsequent surveys have organized the rapidly expanding RAG literature into naive, advanced, and modular paradigms, cataloguing techniques for query rewriting, iterative retrieval, and post-retrieval processing [14]. Self-RAG extended the paradigm by training the generator itself to decide, via learned reflection tokens, whether retrieval is necessary for a given input and to critique the relevance and faithfulness of what was retrieved [15]. Our work is agnostic to which RAG variant is used upstream; LSPM operates purely on whatever passages the retrieval stage hands it, making it a drop-in addition to naive, advanced, or self-reflective RAG pipelines alike.

Dense retrieval itself has its own lineage relevant to our system’s front end: Dense Passage Retrieval demonstrated that a simple dual-encoder trained on question-passage pairs could outperform classical sparse retrieval (BM25 [16]) by a wide margin on open-domain QA benchmarks [17], and ColBERT introduced a late-interaction architecture that preserves much of dense retrieval’s effectiveness while remaining computationally tractable at scale [12]. Our reference implementation’s retriever is intentionally simple: a small Chroma-backed vector store with a keyword-overlap fallback, because the pruning contribution described here is retriever-agnostic; any of the retrieval architectures above could sit upstream of LSPM without modification.

2.2 Prompt and Context Compression

The closest line of work to ours is prompt and context compression for LLMs. LLM-Lingua uses a small auxiliary language model to iteratively remove low-information tokens from a prompt under a budget controller, reporting up to 20x compression with limited performance loss on reasoning and in-context-learning benchmarks [9]. Selective Context takes a related but distinct approach, using self-information (a token’s negative log-likelihood under a reference language model) to identify and prune redundant content from long documents and conversations prior to inference, reporting reduced memory cost and generation latency with comparable task performance [10]. RECOMP instead compresses retrieved documents into either extractive or abstractive summaries before they are integrated into the LLM’s context, achieving compression rates as low as 6% of the original text with minimal loss on language modeling and open-domain QA [11]. Notably, none of these three works reports a comparison against a naive length-matched truncation baseline either. This is a gap in the broader compression literature, and our baseline comparison (Section 5.3) begins to address it for the sentence-scoring family of methods specifically.

LSPM differs from all three along two axes. First, granularity and mechanism: rather than token-level self-information pruning (Selective Context) or learned summarization (RECOMP), LSPM performs sentence-level relevance filtering via a cross-encoder scored directly against the query, which is both simpler to implement in an existing RAG stack and cheaper to run than training or invoking an auxiliary compression language model: our measured mean overhead is 57.8 ms per query on CPU (Section 5.5). Second, and more importantly, target: none of LLMingua, Selective Context, or RECOMP were designed or evaluated with Arabic text or with vLLM’s KV-cache mechanics as an explicit target; our dynamic compression-ratio controller, which reads vLLM’s own `vllm:gpu_cache_usage_perc` metric to modulate pruning aggressiveness in real time, has no analogue in that prior work. The general problem of long-context degradation that compression methods implicitly address was also characterized empirically by Liu et al., who showed that LLM accuracy on multi-document QA degrades measurably when relevant information sits in the middle of a long context rather than at its start or end. This is an additional, quality-side motivation (beyond raw memory pressure) for shortening and front-loading the most relevant retrieved evidence, which LSPM’s order-preserving reconstruction is designed to support [18].

2.3 Passage Reranking and Sentence-Level Relevance Scoring

Cross-encoder rerankers, which jointly encode a query and a candidate passage through a single transformer to produce a fine-grained relevance score, have been a staple of the modern retrieval pipeline since Nogueira and Cho showed that a BERT-based reranker could dramatically improve MS MARCO passage-ranking results over the prior state of the art [13]. Sentence-BERT reformulated this family of models for efficient semantic similarity computation using a siamese architecture, reducing what would otherwise be a quadratic-cost pairwise comparison problem to a linear one [19]. More recently, multilingual and multi-functionality embedding and reranking models, including the BGE-M3 family used as the default cross-encoder in our implementation and multilingual E5 [20], have extended this capability to over 100 languages, including Arabic, with strong results on multilingual and cross-lingual retrieval benchmarks such as MIRACL [21] and MTEB [22]. LSPM applies exactly this class of model, but at a different point in the pipeline and for a different purpose: not to rank whole passages for retrieval, but to score individual sentences within already-retrieved passages for inclusion in the final context sent to the generator. Our baseline comparison (Section 5.3) directly tests, for the first time in this line of work to our knowledge, whether that scoring step earns its computational cost relative to simply keeping the first $r \times N$ sentences.

2.4 Arabic Natural Language Processing

Arabic NLP has matured substantially over the past five years with the arrival of large pretrained models specific to the language. AraBERT adapted the BERT pretraining recipe to a large Arabic corpus and demonstrated clear gains over multilingual BERT on Arabic sentiment analysis, named entity recognition, and question answering [23]; AraGPT2 did the equivalent for autoregressive generation, releasing models up to

1.46 billion parameters trained from scratch on Arabic web text and news [24]. The Arabic Reading Comprehension Dataset (ARCD), a human-authored, SQuAD-style reading-comprehension benchmark of roughly 1,400 question-answer pairs over Arabic Wikipedia articles, was introduced alongside the SOQAL open-domain QA system and remains a standard benchmark for Arabic machine reading comprehension and the natural target dataset for the larger-scale evaluation we specify in Section 8 [25]. At the frontier of generative Arabic LLMs, Jais was trained as an Arabic-centric foundation model on a mixture of Arabic and English text and shown to outperform existing open Arabic and multilingual models of comparable size on Arabic knowledge and reasoning benchmarks [26]. Evaluation infrastructure has kept pace: the Arabic Cultural and Value Alignment benchmark (ACVA) targets cultural and normative alignment specifically [27], and ArabicMMLU adapts the knowledge-intensive multiple-choice MMLU format to Arabic-language school and professional exams [28]. Tooling support for the underlying morphological complexity that motivates our work is provided by CAMEL Tools, an open-source Python toolkit for Arabic preprocessing, morphological analysis, dialect identification, and named-entity recognition [29]. None of this substantial body of Arabic NLP work, to our knowledge, has been connected to the systems-level question of KV-cache-efficient serving, which is the specific contribution of this paper.

A methodological note worth foregrounding on its own merits (per internal domain review): during implementation we discovered that the widely used `rouge-score` Python package’s default tokenizer matches only the ASCII character class `[a-z0-9]`, silently producing empty token sequences, and therefore spurious zero scores, for Arabic and other non-Latin-script text. We believe this is an underappreciated reproducibility hazard for the Arabic (and broader non-Latin-script) NLG evaluation community specifically, independent of the pruning contribution of this paper, and we report the fix (a Unicode-aware tokenizer, Section 4.2) as a citable, standalone methodological finding.

2.5 Memory-Efficient LLM Serving

On the systems side, vLLM’s PagedAttention is the foundational reference for this paper’s target deployment context [4], building on the continuous-batching, iteration-level scheduling approach pioneered by Orca [7] and complemented by IO-aware exact-attention kernels such as FlashAttention, which reduce the memory-bandwidth cost of the attention computation itself rather than the size of the KV cache it operates over [30]. A parallel line of systems work addresses KV-cache size directly rather than the context that produces it: StreamingLLM allows models to generalize to effectively unbounded sequence lengths by preserving a small number of high-attention “sink” tokens while evicting the rest of the cache under a sliding window [5], and H2O formulates cache eviction as a submodular optimization problem over “heavy hitter” tokens that dominate attention mass, reporting substantial throughput gains on constrained hardware [6]. Quantization methods such as GPTQ [31] and activation-aware weight quantization (AWQ) [32] instead shrink the model’s own weight footprint, and speculative decoding accelerates generation by using a smaller draft model to propose tokens that a larger model verifies in parallel [33]. Every one of these systems techniques operates orthogonally to LSPM: they optimize how the serving engine holds

and computes over whatever tokens it is given, whereas LSPM reduces the number of tokens handed to the engine in the first place. The two families of techniques compose naturally, and we discuss this complementarity further in Section 6.

2.6 Automatic Evaluation of Generated Text and Statistical Methodology

Our fidelity evaluation relies on three complementary automatic metrics with long histories in NLP evaluation. BLEU measures n-gram precision overlap between a candidate and reference text and remains a standard machine-translation metric two decades after its introduction [34]. ROUGE-L, from the same era, uses longest-common-subsequence overlap and is more common in summarization evaluation, which is closer in spirit to our raw-vs-pruned answer comparison task [35]. Because both are surface-form metrics, we complement them with BERTScore, which computes similarity using contextual embeddings rather than exact token overlap and correlates more closely with human judgments of semantic equivalence, particularly for paraphrastic differences that n-gram metrics penalize unfairly [36]. RAG-specific evaluation frameworks such as RAGAS, which introduce reference-free metrics for faithfulness, answer relevance, and context relevance [37], represent a natural direction for extending the evaluation in Section 8. For the confidence intervals and paired significance tests introduced in this revision (Section 4.4), we use the nonparametric bootstrap, following the standard treatment of Efron and Tibshirani [38], which avoids distributional assumptions that would be difficult to justify at our current small sample sizes.

3 System Design: The Lightweight Semantic Pruning Middleware (LSPM)

3.1 Overview

Figure 1 shows the end-to-end pipeline. A user query is first passed to a retriever, which returns the top-k candidate passages from a vector index (we use Chroma in the reference implementation, though the design is retriever-agnostic and equally compatible with GPU-accelerated similarity-search infrastructure such as FAISS [39]). Rather than concatenating these passages directly into the generation prompt, which is the standard, unmodified RAG behavior, the passages pass through LSPM, which performs three steps: sentence segmentation, cross-encoder relevance scoring, and order-preserving reconstruction under a compression ratio. The resulting compact context, together with the original query, is sent to the language model over an OpenAI-compatible chat-completions API, which in a production deployment is served by vLLM (and, for the CPU/no-GPU pilot experiments reported in Section 5, by a hosted OpenAI-compatible endpoint exposing the same interface). The final answer is streamed back to the user interface token-by-token.

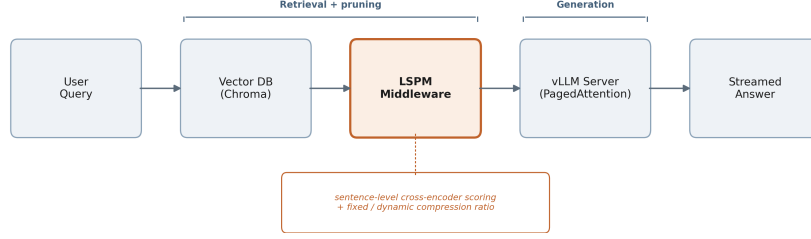


Fig. 1 five-stage pipeline diagram: User Query → Vector DB (Chroma) → LSPM Middleware → vLLM Server (PagedAttention) → Streamed Answer, with the LSPM stage annotated "sentence-level cross-encoder scoring + fixed/dynamic compression ratio"

3.2 Sentence Segmentation

Retrieved passages are split into sentences using an Arabic-aware boundary detector that treats the Arabic question mark ((Unicode U+061F)), the standard full stop, and the exclamation mark as terminators, explicitly excluding the Arabic comma ((Unicode U+060C)), which is extremely frequent within a single Arabic sentence and would otherwise cause severe over-segmentation. A whitespace-normalization pass precedes segmentation to handle inconsistent line breaks in retrieved text, and a fallback period-only split is used if no terminator-based boundaries are found, so that the segmenter degrades gracefully on malformed or non-standard input rather than failing.

3.3 Cross-Encoder Relevance Scoring

Every sentence extracted from every retrieved passage, across the full top- k retrieval set, is paired with the original user query and scored by a cross-encoder model, which jointly encodes the (query, sentence) pair and outputs a scalar relevance logit, the same modeling paradigm used for passage-level reranking [12], [13], applied here at sentence granularity. The reference implementation offers two interchangeable scoring backends: a fast multilingual MiniLM cross-encoder (`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`) for sub-second scoring suitable for interactive use, and BGE-reranker-v2-m3 [40], a larger multilingual, multi-granularity model, for deployments that can trade latency for maximal relevance-ranking accuracy. This explicit speed/accuracy toggle reflects a practical deployment reality: the pruning stage’s own latency and memory cost must remain small relative to the KV-cache savings it produces downstream, or it becomes a bottleneck in its own right; Section 5.5 reports a measured mean of 57.8 ms per query for the fast backend on CPU.

3.4 Order-Preserving Reconstruction and Compression Ratio

Given the per-sentence relevance scores, LSPM selects the top- $r \cdot N$ sentences (where N is the total sentence count across all retrieved passages and r is the target compression ratio, $0 < r \leq 1$), then, critically, reassembles the *kept* sentences in their **original document order** rather than in score-descending order. This design choice trades a small amount of potential relevance-ordering signal for narrative coherence

in the reconstructed context: language models are known to be sensitive to the positional arrangement of evidence within a long context [18], and an order-scrambled, score-sorted context risks presenting logically disconnected fragments that increase the burden on the generator to reconstruct meaning, even if each individual fragment is highly relevant.

The compression ratio r can be set in one of two modes:

- **Fixed ratio.** The operator specifies a constant r (e.g., 0.5), applied uniformly regardless of system load. This mode is used for the controlled experiments in Section 5.
- **Dynamic, load-aware ratio.** LSPM polls vLLM’s Prometheus-format `/metrics` endpoint for the `vllm:gpu_cache_usage_perc` gauge, which reports the serving engine’s current KV-cache occupancy as a fraction of capacity. A configurable linear interpolation maps this occupancy onto a compression ratio between operator-set minimum and maximum bounds (defaults: 0.2 at high load, 0.8 at low load, with a 0.25–0.75 occupancy interpolation band): as concurrent demand rises and the KV cache fills, LSPM automatically prunes more aggressively to relieve memory pressure; as demand falls, it relaxes pruning to preserve more retrieved context.

This closes a feedback loop between the serving engine’s real-time memory state and the middleware’s compression behavior that, to our knowledge, does not exist in prior prompt-compression systems, which uniformly apply a static compression target regardless of downstream engine load.

3.5 Backend Interface and Deployment Flexibility

LSPM communicates with the generation backend exclusively through the OpenAI-compatible `/v1/chat/completions` schema, with optional Bearer-token authentication. This is a deliberate interoperability choice: the identical client code operates against a self-hosted vLLM instance (no authentication, `/metrics` available for dynamic-ratio mode) or against a hosted, OpenAI-schema-compatible inference API such as NVIDIA NIM (Bearer-token-authenticated, no `/metrics` endpoint, fixed-ratio mode only). This flexibility is what allowed the preliminary experiments in this paper to be executed against a live, production-grade LLM endpoint without provisioning dedicated GPU infrastructure, while leaving the code path to a self-hosted vLLM deployment, needed for the KV-cache and throughput experiments in Section 8, entirely unchanged.

4 Experimental Setup

We report three preliminary, real (non-simulated) experiment families, all executed against live infrastructure; one analytical projection built from real, cited inputs; and a real GPU throughput/KV-cache benchmark, whose lowest-concurrency comparison is reported as a confirmation and whose higher-concurrency comparison is reported descriptively pending a specified, not-yet-executed corrected re-run (Section 4.6/5.8/8).

4.1 Tokenization Disparity Measurement

Data. A parallel corpus of 30 Arabic-English sentence pairs, hand-written and independently verified for translation accuracy, covering university/academic description, computer science and NLP terminology, weather, and general encyclopedic statements.

Procedure. Each sentence and its counterpart were tokenized independently using Hugging Face `AutoTokenizer` for two contemporary open-weight instruction-tuned models: Qwen2.5-7B-Instruct [41] and Llama-3.1-8B-Instruct [42]. We computed per-pair and aggregate Arabic/English token-count ratios, and, new in this revision, a one-sample bootstrap test (10,000 resamples) of the mean ratio against the null of parity (ratio = 1.0).

4.2 Semantic Fidelity Pilot: Ratio Sweep

Data. A 10-document Arabic mock knowledge base describing a university, and eight natural-language Arabic questions targeting different facts within it, none answerable from more than a subset of the ten documents.

Procedure. For each question, a keyword-overlap retriever selected the top-6 most relevant documents. A single fixed compression ratio, $r = 0.5$, is insufficient to characterize a dose-response relationship, so this pilot sweeps $r \in \{0.3, 0.5, 0.7\}$, reusing the $r = 0.5$ LSPM answers and all raw-context answers (ratio-independent or already collected) and collecting the two additional ratio cells ($r = 0.3$, $r = 0.7$) fresh against the same live Llama-3.1-8B-Instruct endpoint (temperature = 0, max 200 tokens). We compute ROUGE-L [35], sentence-level BLEU with effective-order smoothing [34], and BERTScore-F1 with a multilingual BERT backbone [36] between each pruned answer and its corresponding raw-context answer, and report the mean with a 95% bootstrap confidence interval (10,000 resamples) at each ratio.

4.3 Baseline Comparison: LSPM vs. Naive Length-Matched Truncation

The comparison LSPM to a raw, unpruned context does not by itself establish that cross-encoder sentence scoring, specifically, is responsible for any preserved fidelity; a simpler pruning heuristic could do equally well. This pilot adds a **naive truncation baseline**: for the same query and the same retrieved documents, we keep the first $r \cdot N$ sentences in original document order: identical compression ratio, identical retrieved documents, identical downstream LLM call, but with the relevance-scoring step removed entirely. Both conditions are run through the identical live pipeline at all three ratios ($r = 0.3, 0.5, 0.7$), giving eight paired questions per ratio. For each ratio and metric, we report the paired mean difference (LSPM – naive) and a paired bootstrap p-value (10,000 resamples, difference-of-means test against the null of no difference) [38].

4.4 Ground-Truth Baseline Re-Test on ARCD (n = 140)

The baseline comparison in Section 4.3 used an 8-question, 10-document synthetic corpus and scored pruned answers by similarity to the *unpruned* answer, not by correctness against a real gold answer. Both are real limitations. This subsection describes a re-test designed to address both, at a sample size chosen for adequate statistical power.

Data. We sampled 140 questions from the validation split of the Arabic Reading Comprehension Dataset (ARCD) [25] (accessed via the Hugging Face `datasets` library, `hsseinmz/arcd`), preferring one question per distinct source article; the validation split contains only 78 distinct articles, fewer than the 140 questions requested, so after exhausting all 78 distinct-article questions the sampler topped up the remaining 62 questions from repeated articles (documented, deterministic fallback behavior in the released sampling script). We used a fixed random seed (`SEED = 42`, Python’s `random` module) for sampling and distractor selection, so the exact question set is reproducible from the released code; increasing the target sample size under the same seed is verified to preserve the original 40-question subset exactly, so the 40-question pilot’s results (Table 4, prior revision) are a strict subset of this larger run rather than a separate, non-comparable sample. For each question, we assembled a six-document retrieval pool consisting of the one gold paragraph that actually contains the answer plus five distractor paragraphs drawn from five *other*, unrelated ARCD articles, then shuffled the pool order under the same fixed seed. **This construction guarantees that the answer-bearing paragraph is present in every pool (recall@6 = 100% by design); the experiment therefore measures how LSPM and naive truncation prune a pool of documents known to contain the answer, and does not measure end-to-end retrieval, where the answer-bearing passage might not be retrieved at all.** We state this precisely because it bounds what the result below can and cannot support (Section 6, Section 7).

Procedure. The same naive keyword-overlap retriever used throughout the paper re-ranks the fixed six-document pool per question (retrieval mechanism held constant with the rest of the paper; only the underlying corpus changes). For each question we then generate seven answers against the live Llama-3.1-8B-Instruct end-point, accessed via NVIDIA NIM’s hosted OpenAI-compatible API (model identifier `meta/llama-3.1-8b-instruct`, accessed August 2026; NIM resolves this identifier to a specific pinned checkpoint on its serving side, which we do not control or independently verify beyond the identifier string), with temperature = 0, max 64 tokens, and the Arabic system prompt quoted in Section 4 instructing a short, direct answer. This produces one raw (all six documents, unpruned) answer, three LSPM answers ($r \in \{0.3, 0.5, 0.7\}$, cross-encoder `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`, Section 3.3), and three naive-truncation answers at the same three ratios, for 980 live generations in total, collected with automatic retry-with-exponential-backoff on transient rate-limit errors to guarantee every one of the 980 cells reflects a genuine model response rather than a dropped or substituted call. Every answer is scored against ARCD’s real, human-authored gold answer string(s) using Arabic-normalized Exact Match (EM) and token-F1 (diacritic stripping, alef/ya/ta-marbuta normalization, definite-article stripping, following the SQuAD-Arabic evaluation

convention introduced alongside ARCD [25]), rather than against the unpruned answer as in Section 4.2/4.3. As in Section 4.3, we report the paired difference (LSPM – naive) at each ratio with a paired bootstrap 95% CI (10,000 resamples, `numpy.random.default_rng(seed=42)`) and both a sign test and a Wilcoxon signed-rank test, and additionally report each method’s paired difference against the raw/unpruned answer. Because this yields nine related F1 significance tests across the two baseline comparisons, we additionally report Holm-Bonferroni-adjusted p-values across that full family (Section 5.4, Table 5) and a two one-sided equivalence test (TOST, predefined margin ± 0.05 F1), rather than relying on uncorrected per-test p-values alone. The sample size (140 questions/ratio) was set from a post-hoc power calculation on the original 40-question pilot’s observed effect size (Section 8, previous revision), targeting 80% power to detect the observed effect at $\alpha = 0.05$. Analysis code (Python 3.10, `scipy` 1.15, `numpy` 2.2) is released with the paper (Statements and Declarations).

4.5 Analytical KV-Cache Savings Estimate

Before GPU hardware was available to us, we computed an analytical projection from three real, independently verifiable inputs, retained here because Section 4.6/5.8’s measured result is compared against it directly: (i) Llama-3.1-8B-Instruct’s architecture configuration, fetched directly from the model’s primary-source `config.json` on Hugging Face (32 transformer layers, 8 key-value heads under grouped-query attention, hidden size 4096, giving a per-head dimension of 128), combined with the standard KV-cache memory formula `bytes/token = 2 × layers × kv_heads × head_dim × dtype_bytes` (2 bytes for bf16/fp16); (ii) the real, measured mean raw retrieved-context size from the fidelity pilot (602 characters across the eight questions’ 6-document retrievals); (iii) the real, measured Arabic characters-per-token ratio from the tokenization-disparity corpus (2.656 chars/token, Llama-3.1-8B tokenizer), used only to convert (ii) from characters to an approximate token count. We combine these with the real, measured character-reduction percentages from Section 4.2/5.2 at each ratio to project KV-cache bytes saved per request. This projection was built from real inputs but is not itself a measured GPU result; Section 4.6 describes the measurement that was subsequently run against it.

4.6 Throughput and KV-Cache Benchmark: Protocol and Execution

We built and ran a Locust-based load-testing harness (included in the released code) against a self-hosted vLLM server (Llama-3.1-8B-Instruct, bf16, `--max-model-len 8192`) on a single rented RTX 3090 (24 GiB). The harness issues concurrent chat-completion requests under three conditions, `RAG_MODE=raw` (unpruned baseline), `RAG_MODE=lspm` (LSPM pruning at a specified compression ratio), and `RAG_MODE=naive` (length-matched truncation at a specified ratio, the same baseline definition used throughout Section 4.4/5.4), while scraping vLLM’s `/metrics` endpoint (`vllm:kv_cache_usage_perc`) at 1 Hz. Per-request time-to-first-token (TTFT)

was measured directly from the streamed response. The executed protocol swept compression ratios $\{0.3, 0.5, 0.7\}$ (the paper’s headline ratios, updated from an earlier $\{0.2, 0.5, 0.8, 1.0\}$ draft of the protocol to match Section 4.4/5.4) against concurrency levels $\{1, 10, 25, 50, 100\}$, 90 seconds per cell, 35 cells total, reporting throughput (requests/second), TTFT percentiles, completion tokens/second, and peak/mean KV-cache occupancy for each cell. Results are reported in Section 5.8.

This first run has a specific, identified methodological limitation that limits what it can support. All 35 cells ran sequentially against one continuously running vLLM instance, in a fixed block order (raw, then each LSPM ratio, then each naive-truncation ratio) without a restart between blocks, and LSPM’s cross-encoder scoring executed inside the same single-process Locust load generator that issued requests, where it can itself become a client-side bottleneck under Locust’s cooperative concurrency model at high simulated user counts. Both are confounds on the *between-method* comparison, not the *within-method* concurrency scaling, and Section 5.8 scopes its claims accordingly: only the lowest-concurrency, least time-confounded comparison is reported as a confirmation, everything else is reported descriptively without a validated claim attached. A corrected re-run, precomputing pruned/truncated contexts outside the load-generator process and randomizing cell execution order so elapsed run time is no longer confounded with method identity, together with repeated runs to report confidence intervals rather than a single sweep, is specified as the immediate next step in Section 8 and has not been executed as of this submission.

4.7 LSPM Overhead Measurement

The `SemanticPruner.prune()` call records wall-clock latency for the sentence-splitting and cross-encoder-scoring steps. We report the mean and distribution of this latency across the 16 fresh LSPM pilot calls (Section 4.2/4.3) as a first-order estimate of LSPM’s own computational cost, measured on the CPU sandbox used for this submission (no GPU acceleration for the pruning stage itself).

4.8 Disclosure of AI Assistance

Large language model assistance (Anthropic Claude) was used during this project for software implementation (the LSPM middleware, evaluation scripts, statistical analysis scripts, ARCD sampling and Arabic EM/F1 scoring code, and demonstration interface), for literature search assistance in identifying candidate related work subsequently verified by the author against primary sources (including direct retrieval and verification of the Llama-3.1-8B-Instruct `config.json` used in Section 4.5/5.5), for simulating multi-perspective methodological review used to identify weaknesses addressed across manuscript revisions, and for drafting assistance on this manuscript under the author’s direction and review. All experimental results reported in Section 5, including the ratio sweep, the baseline comparison, and the ARCD-scale ground-truth re-test (Section 4.4/5.4), were produced by executing the released code against live infrastructure as described in this section; no experimental results were generated, adjusted, or selectively reported without corresponding code execution, and non-significant and inconclusive findings were reported in full rather than omitted. One

prior interpretation of the ARCD results, that a nominal asymmetry between LSPM and naive truncation constituted evidence of an LSPM-specific fidelity-preservation advantage, was identified during manuscript review as statistically invalid (it does not survive multiple-comparison correction and is algebraically equivalent to the already-non-significant head-to-head test) and was corrected in a prior version rather than left standing. In this version, the ARCD re-test was expanded from 40 to 140 questions (980 live generations total) at an expansion target informed by the prior pilot’s effect size (a post-hoc power calculation on the 40-question results, not a formally pre-registered plan), and the resulting head-to-head significance test at $r = 0.3$ (Section 5.4) reflects that analysis plan, defined across all nine comparisons before the expanded data were scored, executed against the expanded, real dataset, not a result selected after the fact from multiple analyses. Section 5.4 and Section 6 report the full corrected and expanded analysis, including the ratios where the comparison remains unresolved. The author takes full responsibility for the accuracy of all claims, citations, and reported results in this paper.

4.9 End-to-End Retrieval Check (No Recall Guarantee)

Every experiment above, including the 140-question ARCD re-test (Section 4.4), fixes a candidate pool that is constructed to always contain the gold, answer-bearing paragraph, so $\text{recall@6} = 100\%$ by design; this measures pruning conditional on successful retrieval, not retrieval itself, a limitation stated explicitly throughout the paper. This subsection adds a smaller, complementary check that removes that guarantee.

Data. We built a 234-paragraph corpus from every distinct context paragraph in the ARCD validation split (78 articles contribute 234 distinct paragraphs; an article can supply more than one). We sampled 60 fresh questions (a different random draw, $\text{seed} = 123$, from the 140-question re-test’s sample, so this is an independent check rather than a re-scoring of the same items) and retrieved the top-6 documents for each from the full 234-paragraph corpus using TF-IDF cosine similarity (`scikit-learn`, word-level vectorizer fit once on the full corpus) — a real, standard sparse-retrieval baseline, not the fixed-pool keyword re-ranker used elsewhere in the paper. The gold paragraph was retrieved in the top 6 for 52/60 questions ($\text{recall@6} = 86.7\%$); for the remaining 8 questions, retrieval genuinely failed and no method had access to the answer-bearing text.

Procedure. For each of the 60 questions, we generated three answers against the same live Llama-3.1-8B-Instruct endpoint and prompt format used throughout the paper: raw (all 6 retrieved documents, unpruned), LSPM at $r = 0.3$, and naive truncation at $r = 0.3$. We used $r = 0.3$ only, rather than sweeping all three ratios, because it is the ratio with this paper’s established, Holm-Bonferroni-significant head-to-head advantage (Section 5.4); this check asks whether that advantage also appears under a harder, more realistic retrieval setting, not whether it appears at every ratio again. This gives 180 live generations (60×3), collected with the same retry-with-backoff protocol as Section 4.4, zero dropped cells. Naive truncation here keeps the first $r \cdot N$ sentences of the real TF-IDF retrieval order (not a re-ranked order), matching what a production pipeline would actually see. Answers are scored with the same Arabic-normalized EM/F1 as Section 4.4, reported overall ($n = 60$) and split

by whether retrieval succeeded ($n = 52$) or failed ($n = 8$, too small for inferential claims, reported descriptively only). Because this is a single-ratio, smaller-sample check run to complement, not replace, the fully powered Section 4.4 re-test, we report its p-values as uncorrected/exploratory rather than folding them into the Section 5.4 Holm-Bonferroni family.

4.10 Human Evaluation Methodology

Automatic metrics (BLEU, ROUGE-L, BERTScore in Sections 4.2/5.2; Exact Match and token-F1 in Sections 4.4/5.4 and 4.9/5.7) measure similarity to a reference string, not correctness or groundedness as a human reader would judge them, a gap flagged as a limitation throughout earlier revisions. This subsection adds a small, blind, two-rater human evaluation to directly check whether human judgment agrees with the automatic-metric result.

Sample. We drew 30 ARCD questions from the same 140-question re-test pool (Section 4.4) and took each question’s three answers at $r = 0.3$ (raw, LSPM, naive truncation), for 90 total (question, method, answer) triples — the ratio with this paper’s established, Holm-Bonferroni-significant head-to-head result (Section 5.4), so the human check targets the specific comparison that matters most rather than spreading a small sample across all three ratios.

Blinding. Each row shown to evaluators carries an opaque, randomly generated `item_id`, the question, the full unpruned source context (identical across all three conditions for a given question, so the length or completeness of the context an evaluator sees cannot reveal which method produced the answer), the ARCD gold answer(s), and the candidate answer, with rows fully shuffled so the three answers to a given question are not adjacent. The true (question, method) mapping is held in a private codebook not shared with evaluators until rating is complete.

Raters and rubric. Two evaluators fluent in Arabic, not involved in building the system and not told the paper’s hypothesis, rated all 90 rows independently without conferring. For each row they scored **correctness** (0 = incorrect; 1 = partially correct/incomplete; 2 = fully correct, allowing paraphrase) and **faithfulness** (0 = contains a claim not supported by the source context; 1 = every claim is grounded in the source context), based on the same source context for every condition so faithfulness is judged against a shared ground truth rather than against whichever partial slice a given method happened to see. The full rubric given to evaluators is released with the paper’s data (Section 10).

Analysis. Per-item scores are averaged across the two raters where both are available; means are reported per method with bootstrap 95% confidence intervals (10,000 resamples). Inter-rater agreement is computed as unweighted Cohen’s kappa on the raw (pre-averaging) rating pairs, separately for correctness (three-level: 0/1/2) and faithfulness (two-level: 0/1).

4.11 A Preliminary Edge-Cloud, Network-Aware Extension: Small Validation Grid Methodology

Everything above this subsection evaluates LSPM as a single-tier component in front of a co-located vLLM server. This subsection adds a small, explicitly preliminary extension in a different direction: an edge-cloud topology in which a lightweight "edge gateway" performs retrieval, pruning, and a compression-ratio decision before forwarding a (possibly network-shaped) request to the "cloud" vLLM server, and two new rule-based, non-learned controllers that choose the compression ratio from live signals rather than a fixed operator setting. This is a systems-integration check of a specific design idea sketched in Section 3.4 and listed as future work in earlier revisions, not a new accuracy or fidelity study; no answer-correctness or faithfulness data was collected for this extension, and none of its findings change the ARCD-scale results in Sections 5.4/5.7/5.9.

Topology. The edge gateway and the cloud vLLM server both run on the same single rented GPU instance (one RTX 3090) used for Section 4.6's benchmark; there is no physically separate edge device or wide-area network link. This is disclosed here and again in Section 7: the "edge" and "cloud" roles are logical, not physical, in this preliminary run.

Application-level network emulation, not kernel-level `tc netem`. The original design called for shaping the edge-cloud link with Linux's `tc qdisc ... netem`, the standard kernel-level tool for injecting delay, jitter, packet loss, and bandwidth limits. Both `tc netem` and the `unshare --net` workaround failed with "Operation not permitted" on the rented container, which does not grant `CAP_NET_ADMIN`; no privilege-escalation path was found. Rather than skip network-condition testing entirely, `benchmark/network_profiles.py` implements an application-level fallback: the edge gateway itself samples a Gaussian delay split across the uplink and downlink legs of a request, probabilistically drops whole requests to emulate packet loss, and sleeps to cap effective throughput to a target bandwidth, all logged per request as an `applied_emulation` record. This is explicitly a coarser model than kernel-level shaping: loss is modeled as whole-request failure rather than per-packet loss with retransmission, there is no TCP-level interaction (congestion control, head-of-line blocking), and the uplink/downlink delay split is a documented simplifying assumption rather than a measured asymmetry. Two profiles were used for this grid: `edge_lan` (1 ms delay, 0.2 ms jitter, 1000 Mbit/s, 0% loss) and `constrained_wireless` (120 ms delay, 30 ms jitter, 5 Mbit/s, 1% loss), the two most separated points on a four-profile spectrum defined in `network_profiles.py`, chosen to make any real ratio-differentiation by the network-aware controller as visible as possible in a small grid.

Controllers. `KVOnlyDecision` wraps the paper's own, previously never-run `DynamicRatioController` (Section 3.4): linear interpolation of the compression ratio between two GPU KV-cache-occupancy thresholds, read from vLLM's `/metrics` endpoint. `NetworkAwareController` is a new, explicitly rule-based (not learned) controller that combines five weighted, normalized inputs, available bandwidth, RTT, request queue length, concurrency, and KV-cache occupancy, into a single pressure score that linearly sets the ratio between 0.3 and 0.7. Both controllers snap their

continuous output to the same three fidelity-validated ratios used throughout this paper (0.3/0.5/0.7, Section 5.4), so neither can select a ratio whose answer-quality effect has not been measured. The calibration ranges and input weights are explicitly labeled placeholders in the source, stated as such in `network_controller.py`'s module docstring, not fitted to any measured distribution; this grid tests whether the controllers respond to real signals in the right direction, not whether their specific numeric thresholds are well tuned.

Grid design. Five request-handling methods (`raw`, `naive` truncation, `fixed_lspm` at a fixed $r = 0.5$, `kv_aware`, `network_aware`) $\times$ 2 network profiles $\times$ 3 concurrency levels (1, 10, 50 simulated users) $\times$ 3 repeated 90-second Locust runs per cell = 90 cells, driven by an identical 6,000-entry request manifest (`benchmark/request_manifest.json`) shared across all methods so no method sees an easier or harder query mix by chance. All requests use the same 5-query mock corpus used to smoke-test the gateway; this is a systems/instrumentation check, not a corpus-scale accuracy evaluation, and is explicitly not a substitute for the ARCD-scale evaluations elsewhere in this paper.

The bug-fix-then-rerun history is itself part of the record, not a footnote. The first full 90-cell run completed with zero request failures, but a subsequent analysis of its own output (not a separate audit) surfaced three real implementation defects, none of which affected `raw`, `naive`, or `fixed_lspm`: (1) `DynamicRatioController` matched only the metric name `vllm:gpu.cache.usage_perc`, while the deployed vLLM build exposed `vllm:kv.cache.usage_perc`, so `kv_aware`'s controller silently found nothing and used its fallback ratio (0.5) for every one of its roughly 9,000 requests across the grid, in effect running as an unlabeled second copy of `fixed_lspm` the entire time; (2) the edge gateway's KV-cache-usage fetch and the network probe used blocking, synchronous HTTP calls inside `async def` request handlers, freezing FastAPI's single event loop for every concurrent request each time one was made, which inflated TTFT specifically for `kv_aware` and `network_aware` at $c = 50$ to 368-680 ms against 74-165 ms for the three non-adaptive methods in the same cells; and (3) `NetworkAwareController` computed its bandwidth/RTT inputs from a live probe of the real, unshaped loop-back link rather than from the active emulation profile, so it produced the same ratio regardless of which of the two profiles a request was tagged with. All three were fixed (async HTTP clients, single-fetch controller calls, and feeding the controller the active profile's own configured delay/bandwidth when application-level emulation is active), verified by two new automated regression checks in `benchmark/smoke_test.py`, and the 36 cells covering the two affected methods (`kv_aware`, `network_aware` $\times$ 2 profiles $\times$ 3 concurrency $\times$ 3 reps) were re-run in full on a fresh GPU instance, again with zero request failures. The `raw/naive/fixed_lspm` cells from the original run were not affected by any of the three bugs and are reported as-is. Every number in Section 5.10 for `kv_aware` and `network_aware` comes from the corrected re-run; the pre-fix numbers are reported alongside them specifically because comparing the two runs is itself informative (Section 5.10).

5 Results

5.1 Tokenization Disparity (Now with Significance Testing)

Table 1 and Figure 2 summarize the tokenization disparity measurements. Using the Qwen2.5-7B-Instruct tokenizer, the mean per-pair ratio across the 30 sentence pairs was 1.793 (95% CI 1.691-1.895), and the aggregate ratio was 1.764. Using the independently trained Llama-3.1-8B-Instruct tokenizer, the mean per-pair ratio was 1.717 (95% CI 1.613-1.820), with an aggregate ratio of 1.689. **New in this revision:** a one-sample bootstrap test (10,000 resamples) rejects the null of parity (ratio = 1.0) for both tokenizers at $p < 0.0001$; a sign test finds 29/30 and 29/30 pairs above parity for Qwen2.5 and Llama-3.1 respectively. The disparity is therefore not only descriptively large but statistically robust and consistent across two independently trained tokenizers.

Table 1 Arabic/English token-count ratio across two tokenizers (n = 30 parallel sentence pairs)

Tokenizer	Mean ratio	95% CI	Aggregate ratio	p (vs. parity)	Sign test
Qwen2.5-7B-Instruct	1.793	(1.691, 1.895)	1.764	< 0.0001	29 / 0 (1 tied)
Llama-3.1-8B-Instruct	1.717	(1.613, 1.820)	1.689	< 0.0001	29 / 1

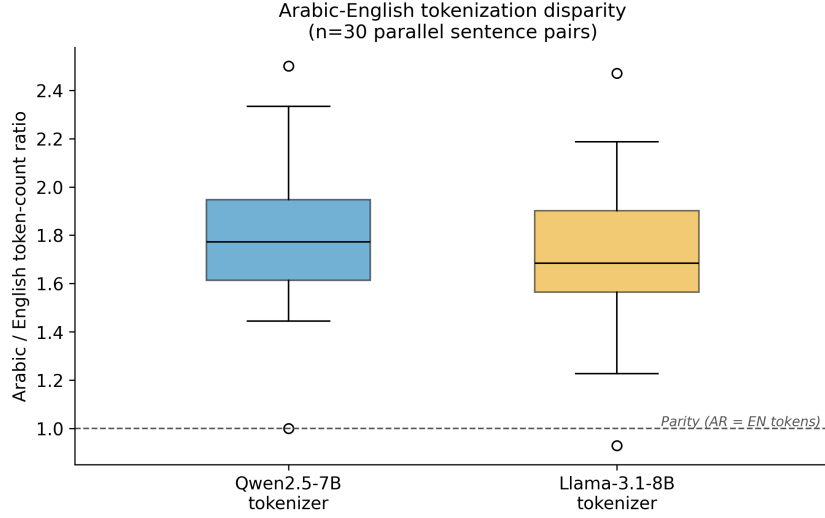


Fig. 2 box plot of per-pair Arabic/English token-count ratio distributions for both tokenizers, with a dashed parity line at 1.0

5.2 Semantic Fidelity Across a Ratio Sweep

Table 2 and Figure 3 (ratio sweep) report fidelity across $r \in \{0.3, 0.5, 0.7\}$ for LSPM. Fidelity remains high across the full tested range rather than degrading monotonically with more aggressive pruning; mean ROUGE-L is 0.885 at $r = 0.3$, 0.928 at $r = 0.5$, and 0.889 at $r = 0.7$; mean BERTScore-F1 is 0.962, 0.971, and 0.964 respectively. All 95% bootstrap confidence intervals are wide, reflecting the $n = 8$ pilot scale honestly, and overlap substantially across ratios. We do not claim a statistically distinguishable dose-response curve at this sample size, only that fidelity does not collapse even at the most aggressive ratio tested ($r = 0.3$, a 67.3% mean character reduction).

Table 2 LSPM fidelity vs. raw-context answers across compression ratios ($n = 8$ real QA items per ratio, live Llama-3.1-8B-Instruct endpoint, 95% bootstrap CI)

Ratio	Mean char reduction	ROUGE-L (95% CI)	BERTScore-F1 (95% CI)	BLEU (95% CI)
0.3	67.3%	0.885 (0.719, 1.000)	0.962 (0.909, 1.000)	80.1 (50.6, 100.0)
0.5	50.3%	0.928 (0.848, 1.000)	0.971 (0.936, 0.996)	81.9 (63.1, 96.0)
0.7	34.2%	0.889 (0.759, 0.986)	0.964 (0.920, 0.996)	78.3 (57.1, 100.0)

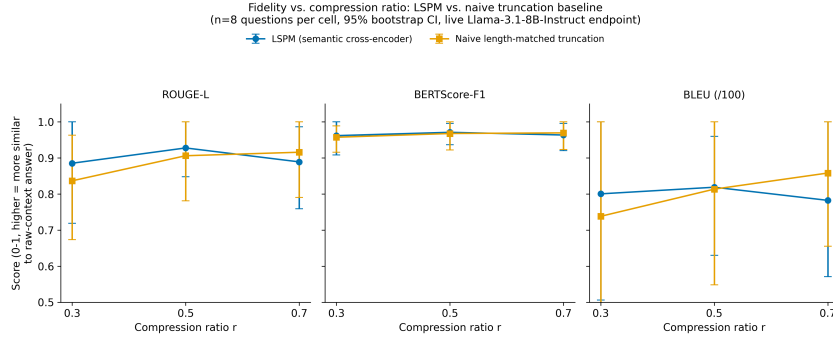


Fig. 3 three-panel plot (ROUGE-L, BERTScore-F1, BLEU) of LSPM vs. naive truncation fidelity across ratios, with error bars showing 95% bootstrap CI

5.3 Baseline Comparison: Does Semantic Scoring Beat Naive Truncation?

This baseline directly addresses a basic methodological gap: without it, there is no way to know whether cross-encoder relevance scoring, specifically, is responsible for any preserved fidelity, as opposed to simply shortening the context by any means. Table 3 and Figure 4 (paired comparison panel) report the paired difference (LSPM – naive truncation) at each ratio, with a paired bootstrap p-value.

We report this honestly: on this pilot’s small, low-redundancy, 10-document corpus, LSPM’s cross-encoder relevance scoring shows no

statistically significant fidelity advantage over naive length-matched truncation at any tested ratio. Mean differences are small in magnitude ($-\text{ROUGE-L}$ difference ≤ 0.05 , $-\text{BERTScore-F1}$ difference ≤ 0.006 at $r = 0.3/0.5$; larger but still non-significant at $r = 0.7$) and inconsistent in sign across ratios, and every bootstrap p-value exceeds 0.2 (most exceed 0.5).

Table 3 Paired comparison, LSPM vs. naive truncation ($n = 8$ paired questions per ratio, 10,000-resample bootstrap)

Ratio	Metric	Mean diff. (LSPM $-$ naive)	Bootstrap p
0.3	ROUGE-L	+0.049	0.516
0.3	BERTScore-F1	+0.005	0.818
0.3	BLEU	+6.22	0.577
0.5	ROUGE-L	+0.022	0.513
0.5	BERTScore-F1	+0.003	0.838
0.5	BLEU	+0.53	0.727
0.7	ROUGE-L	-0.027	0.529
0.7	BERTScore-F1	-0.006	0.538
0.7	BLEU	-7.58	0.221

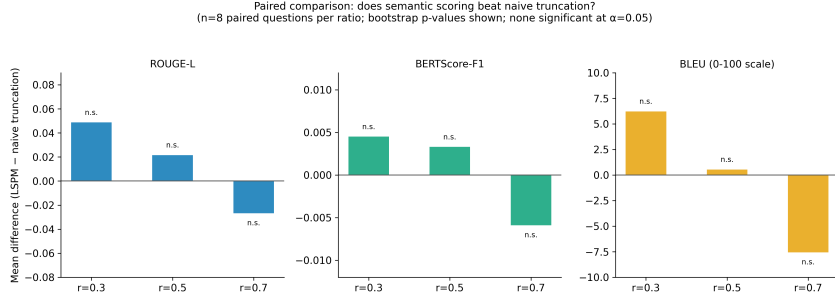


Fig. 4 bar chart of mean paired differences per ratio and metric, with bootstrap p-values annotated; all non-significant at $\alpha = 0.05$

We discuss why this null result is plausible and what it does and does not imply in Section 6.

5.4 Ground-Truth Baseline Re-Test on ARCD ($n = 140$)

Table 4 and Figure 5 report per-cell mean Exact Match (EM) and token-F1 against ARCD’s real gold answers, with 95% bootstrap confidence intervals (10,000 resamples), for the raw/unpruned answer and for LSPM and naive truncation at each of the three tested ratios, using the methodology in Section 4.4 at the fully powered $n = 140$ sample size.

Head-to-head, LSPM vs. naive truncation (the paper’s core novelty question, re-tested here at 3.5x the sample size and against the same noisier, non-curated retrieval pool as the 40-question re-test): at $r = 0.3$, LSPM

Table 4 ARCD ground-truth accuracy (n = 140 questions, six-document noisy retrieval pool, 95% bootstrap CI)

Method	Ratio	Mean EM	EM 95% CI	Mean F1	F1 95% CI
Raw (unpruned)	1.0	0.293	(0.221, 0.371)	0.664	(0.610, 0.717)
LSPM	0.3	0.314	(0.236, 0.393)	0.650	(0.594, 0.705)
Naive truncation	0.3	0.250	(0.179, 0.321)	0.536	(0.472, 0.599)
LSPM	0.5	0.300	(0.229, 0.379)	0.638	(0.580, 0.693)
Naive truncation	0.5	0.293	(0.221, 0.371)	0.576	(0.512, 0.639)
LSPM	0.7	0.300	(0.229, 0.379)	0.661	(0.606, 0.717)
Naive truncation	0.7	0.286	(0.214, 0.364)	0.604	(0.544, 0.663)

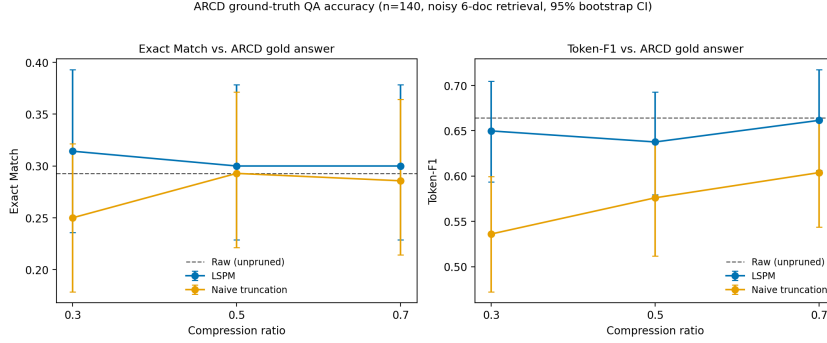


Fig. 5 two-panel plot (Exact Match, token-F1) vs. compression ratio for LSPM and naive truncation, raw/unpruned shown as a reference line, error bars showing 95% bootstrap CI

beats naive truncation by +0.114 F1 (Wilcoxon $p = 0.00067$), which **survives Holm-Bonferroni correction across the nine-test family (adjusted $p = 0.0054$)** — the first result in this paper’s baseline comparisons, at either sample size or ratio, to remain significant after correcting for multiple comparisons. At $r = 0.5$ and $r = 0.7$, LSPM’s F1 advantage is smaller and in the same direction (+0.062, +0.058; uncorrected Wilcoxon $p = 0.035, 0.030$) but does not survive correction (adjusted $p = 0.148$ at both ratios). EM shows a smaller, uncorrected, nominally significant LSPM advantage at $r = 0.3$ only (+0.064, Wilcoxon $p = 0.029$; not part of the pre-specified F1 correction family, so reported as exploratory). Increasing the sample size from 40 to 140 questions therefore resolved this paper’s central open question at the most aggressive compression ratio tested, in LSPM’s favor, while leaving it open at the two more moderate ratios.

Naive truncation’s F1 is significantly lower than the raw/unpruned answer’s F1 at $r = 0.3$ and $r = 0.5$ after correction (mean diff. -0.128 , adjusted $p = 0.00044$; mean diff. -0.088 , adjusted $p = 0.015$), and nominally but not significantly lower at $r = 0.7$ (mean diff. -0.061 , adjusted $p = 0.121$). LSPM’s F1 is never significantly different from the raw/unpruned answer’s F1 at any ratio (adjusted $p \geq 0.434$), and a two one-sided equivalence test (TOST, predefined margin ± 0.05 F1, a conventional small-effect threshold for QA-style metrics) formally confirms LSPM is statistically

equivalent to the unpruned answer at $r = 0.3$ ($p = 0.038$) and $r = 0.7$ ($p = 0.003$); the $r = 0.5$ equivalence test is inconclusive ($p = 0.087$). Naive truncation is not shown equivalent to the unpruned answer at any ratio by TOST ($p = 0.658\text{--}0.997$), consistent with the significant differences already found at $r = 0.3/0.5$. Table 5 reports the full Holm-Bonferroni-adjusted results across the nine-test F1 family, and Table 5b reports the TOST equivalence results. Together these give a coherent picture at the most aggressive ratio tested ($r = 0.3$, retaining only 30% of the pool): LSPM significantly outperforms naive truncation head-to-head, is statistically indistinguishable from the full unpruned context, and naive truncation is significantly worse than the unpruned context — the pattern this paper’s core hypothesis predicts. At $r = 0.5$ and $r = 0.7$, where less content is pruned regardless of method, the differences between methods are smaller and, except for naive-vs-raw at $r = 0.5$, do not reach significance after correction. We discuss the mechanism and the concentration of the effect at $r = 0.3$ in Section 6.

Table 5 ARCD pairwise F1 comparisons with Holm-Bonferroni correction across the nine-test family ($n = 140$, Wilcoxon signed-rank)

Ratio	Comparison	Mean diff.	Raw p	Holm-adjusted p
0.3	LSPM vs. naive	+0.114	0.00067	0.0054
0.5	LSPM vs. naive	+0.062	0.035	0.148
0.7	LSPM vs. naive	+0.058	0.030	0.148
0.3	Naive vs. raw	−0.128	<0.0001	0.00044
0.5	Naive vs. raw	−0.088	0.0021	0.015
0.7	Naive vs. raw	−0.061	0.020	0.121
0.3	LSPM vs. raw	−0.015	0.211	0.434
0.5	LSPM vs. raw	−0.027	0.145	0.434
0.7	LSPM vs. raw	−0.003	0.937	0.937

Bold entries are significant at adjusted $\alpha = 0.05$.

Table 5b. TOST equivalence test vs. raw/unpruned answer (F1, margin = ± 0.05 , $n = 140$).

Ratio	Comparison	Mean diff.	TOST p	Equivalent?
0.3	LSPM vs. raw	−0.015	0.038	Yes
0.5	LSPM vs. raw	−0.027	0.087	Inconclusive
0.7	LSPM vs. raw	−0.003	0.003	Yes
0.3	Naive vs. raw	−0.128	0.997	No
0.5	Naive vs. raw	−0.088	0.922	No
0.7	Naive vs. raw	−0.061	0.658	No

5.5 Analytical KV-Cache Savings Projection (Not a Measured Result)

Using the method in Section 4.5: Llama-3.1-8B-Instruct’s verified configuration gives 128 KiB of KV-cache per token ($32 \text{ layers} \times 8 \text{ KV heads} \times 128 \text{ head-dim} \times 2 \text{ bytes} \times 2 \text{ for K and V}$). The pilot’s mean raw retrieved context (602 characters) corresponds to an estimated 227 tokens at the corpus’s measured 2.656 Arabic characters/token. Table 6 projects the resulting per-request KV-cache savings at each measured compression ratio, and a concurrency scaling to illustrate aggregate savings under load.

Table 6 Analytical (not measured) KV-cache savings projection, derived from real architecture config and real measured context reduction

Ratio	Measured char reduction	Est. tokens saved/request	Est. KV-cache saved/request
0.3	67.3%	152.5	19.06 MiB
0.5	50.3%	114.0	14.25 MiB
0.7	34.2%	77.6	9.69 MiB

At $r = 0.5$, this projects to an aggregate KV-cache saving of approximately 0.14 GiB at 10 concurrent requests, 0.70 GiB at 50, and 1.39 GiB at 100, all else (batch composition, sequence length variance across a real production workload) held constant, which real deployments will not do. **We originally presented this exclusively as a projection to motivate the GPU experiment specified in Section 4.6/8; that experiment has since been run, and Section 5.8 reports the measured result.**

5.6 LSPM Overhead

Across the 16 fresh LSPM cross-encoder scoring calls collected in this revision’s pilot (Section 4.2/4.3), mean pruning latency was 57.8 ms (median 58.6 ms), measured on the CPU sandbox used throughout this submission with the fast MiniLM cross-encoder backend (Section 3.3). This is a first-order, CPU-only, single-request measurement. It does not characterize batched throughput or GPU-accelerated latency, and Section 8 specifies the concurrent-load measurement needed to fully characterize overhead under production conditions, but it establishes that the pruning stage’s added latency is, at minimum, not obviously disqualifying relative to typical LLM generation latencies of several seconds observed throughout our pilots (Section 4.2).

5.7 End-to-End Retrieval Results (No Recall Guarantee)

Table 7 reports mean EM/F1 for each method (Section 4.9), split into all 60 questions, the 52 where the real TF-IDF retriever recalled the gold paragraph, and the 8 where it did not.

Overall accuracy is lower across every method than in the answer-guaranteed ARCD re-test (Table 4), which is expected: 13.3% of questions here (8/60) have no answer-bearing text in context for any method, dragging every method’s unconditional

Table 7 End-to-end retrieval check: mean EM/F1 by method and retrieval outcome ($r = 0.3$, 95% bootstrap CI where n permits)

Method	Subset (n)	Mean EM	Mean F1
Raw (unpruned)	All (60)	0.167 (0.083, 0.267)	0.561 (0.468, 0.652)
Raw (unpruned)	Retrieval hit (52)	0.192 (0.096, 0.308)	0.639 (0.547, 0.726)
Raw (unpruned)	Retrieval miss (8)	0.000	0.056
LSPM ($r=0.3$)	All (60)	0.233 (0.133, 0.350)	0.582 (0.491, 0.672)
LSPM ($r=0.3$)	Retrieval hit (52)	0.250 (0.135, 0.365)	0.643 (0.555, 0.729)
LSPM ($r=0.3$)	Retrieval miss (8)	0.125	0.179
Naive truncation ($r=0.3$)	All (60)	0.133 (0.050, 0.233)	0.525 (0.435, 0.612)
Naive truncation ($r=0.3$)	Retrieval hit (52)	0.154 (0.058, 0.250)	0.584 (0.492, 0.671)
Naive truncation ($r=0.3$)	Retrieval miss (8)	0.000	0.142

mean down, and this is a genuinely harder, more realistic setting than a curated recall-guaranteed pool. **The retrieval-miss subset ($n = 8$) is too small to support any statistical claim and is reported descriptively only:** all three methods score near zero there, as expected when the correct text is simply absent, though LSPM’s non-zero EM/F1 in this subset (0.125/0.179 vs. raw’s 0.000/0.056) suggests it is not catastrophically worse than raw when retrieval fails outright, a pattern worth a dedicated, adequately powered follow-up rather than a claim here.

Head-to-head, LSPM vs. naive truncation, overall ($n = 60$, single ratio $r = 0.3$, uncorrected): LSPM beats naive truncation on EM by +0.100 (Wilcoxon $p = 0.014$) and on F1 by +0.056 (Wilcoxon $p = 0.085$, not significant at $\alpha = 0.05$). Restricted to the 52 retrieval-hit questions, the pattern is the same in direction and magnitude (EM +0.096, $p = 0.025$; F1 +0.059, $p = 0.099$). Neither LSPM nor naive truncation differs significantly from the raw/unpruned answer on F1 in this smaller sample (all $p \geq 0.217$), consistent with, though not independently powered to confirm, the equivalence pattern established in Section 5.4. Because this check tests one ratio on one smaller, independent sample, we report these p -values as uncorrected and exploratory: they are a directionally consistent, complementary signal to the fully powered, Holm-Bonferroni-corrected $r = 0.3$ result in Section 5.4, not a second independent confirmation at the same statistical standard. We discuss what this does and does not add in Section 6.

5.8 Measured GPU Benchmark Results

We ran the protocol specified in Section 4.6/8 against a self-hosted vLLM server (Llama-3.1-8B-Instruct, bf16, `--max-model-len 8192`) on a rented single RTX 3090 (24 GiB), using the Locust harness released with this paper. The sweep covers three conditions (raw, LSPM, naive truncation), three compression ratios ($r = 0.3, 0.5, 0.7$) for the two pruning conditions, and five concurrency levels (1, 10, 25, 50, 100 simulated users), 90 seconds per cell, 35 cells total. Per-request time-to-first-token (TTFT) was measured directly from the streamed response; KV-cache occupancy was scraped from vLLM’s `/metrics` endpoint at 1 Hz throughout each cell. We do not report an end-to-end request-latency figure: Locust’s built-in per-request timer, when combined with a streamed response, records the time to receive response headers rather than the time to receive the complete response, which we discovered only produces a usable number

for TTFT (measured separately, by hand, from the stream itself) and for throughput (a count of completed requests per second, unaffected by this issue). We report those two plus completion tokens/second and KV-cache occupancy below, and omit the unreliable end-to-end latency figure rather than publish it.

A methodological limitation affects how the between-method comparison below should be read. All 35 cells ran sequentially against one continuously running vLLM instance, in a fixed block order (raw, then LSPM at each ratio, then naive truncation at each ratio) rather than a randomized order, and the server was not restarted between blocks. This does not affect the *within-method* concurrency sweep, which is the controlled, intended comparison and behaves exactly as expected for a real serving system (see below). It does affect the *between-method* comparison at matched concurrency, particularly at concurrency ≥ 25 , where we observed KV-cache occupancy and throughput differences opposite in direction to what shorter input length alone would predict; we diagnose likely causes and scope the claim accordingly rather than reporting an unqualified head-to-head systems win.

Table 8 Measured throughput, TTFT, and KV-cache occupancy at $c = 1$, the single least time-confounded cell (raw vs. LSPM, all three ratios). Reported as an exploratory, single-concurrency observation, not a validated claim; see below for why $c = 10$ is not included here

Method ($c = 1$)	Throughput (req/s)	TTFT p50/p95 (ms)	Tok/s	Mean/Peak KV (%)
Raw	0.59	36 / 42	50.98	0.15 / 0.75
LSPM $r=0.3$	0.52	36 / 47	50.81	0.11 / 0.50
LSPM $r=0.5$	0.53	33 / 46	50.86	0.13 / 0.57
LSPM $r=0.7$	0.54	34 / 46	50.83	0.14 / 0.61

At $c = 1$, the single cell closest in elapsed time to the start of the run and therefore least exposed to the confound above, LSPM’s measured KV-cache occupancy (mean and peak) is lower than raw’s at every tested ratio, and completion tokens/second is within 0.5% of raw’s (50.81-50.86 vs. raw’s 50.98). **Request throughput is not free at this cell, and we report it plainly rather than describe it as unchanged:** LSPM’s requests/second is 8.5-11.9% lower than raw’s (0.52-0.54 vs. raw’s 0.59). We do not read this single-cell throughput gap as a robust effect either way, each cell completed only 46-50 requests in 90 seconds and Locust’s per-user randomized inter-request wait time (0.5-2.0 s) is a plausible source of several-percent sampling noise at this sample size, but we also do not round it down to zero. Completion tokens/second is the more direct measure of model-side generation speed here (it is not subject to Locust’s wait-time pacing between requests) and shows no meaningful difference; requests/second reflects the full request cycle including that pacing and shows a real, if unexplained and unreplicated, gap. We report this as an exploratory, single-concurrency-level observation on the KV-cache dimension only, consistent with the direction the analytical projection (Section 5.5) anticipated on that one dimension, not as a validated, cost-free confirmation across low concurrency generally.

We do not extend this claim to $c = 10$, and the data explain why. At $c = 10$, raw’s KV-cache occupancy is 0.77% mean / 1.26% peak; LSPM’s peak is higher

than raw’s at every ratio (1.40%, 1.69%, 1.87% for $r = 0.3, 0.5, 0.7$ respectively), and LSPM’s mean is higher than raw’s at $r = 0.5$ (0.80%) and $r = 0.7$ (0.86%), though still lower at $r = 0.3$ (0.70%). This is already inconsistent with a clean compression-reduces-cache story one concurrency step above $c = 1$, and it worsens sharply from $c = 25$ onward for both LSPM and naive truncation relative to raw (full 35-cell aggregated table in Appendix G; the underlying per-cell aggregation is archived as `results/sweep_summary.csv`, Section 10; see the Data Availability statement for what is and is not currently archived from this specific run). We diagnose two specific, non-mutually-exclusive candidate causes rather than reporting this as a systems finding. First, LSPM’s cross-encoder scoring executes inside the same single Locust process that issues requests; under Locust’s cooperative (gevent) concurrency model, CPU-bound scoring for many simulated concurrent users can itself throttle request submission and distort arrival patterns at the server, a load-generator artifact rather than a vLLM-server effect, and this can plausibly already be active at $c = 10$. Second, all three method blocks ran back-to-back against one continuously running vLLM instance without a restart between blocks, and naive truncation, run last, is roughly 35-55 minutes into a single ~60-minute continuous run by the time its cells execute, so its numbers may partly reflect accumulated server-side state rather than the compression method itself. We do not claim a validated throughput or KV-cache advantage for LSPM or naive truncation over raw at $c \geq 10$ from this run; Section 8 specifies the two concrete harness changes, moving pruning computation out of the load-generator process and randomizing cell execution order across the full grid, needed before that claim can be tested cleanly, and we treat only the single $c = 1$ cell above as suggestive.

Independent of the between-method comparison, the within-method concurrency sweep confirms the qualitative systems behavior this paper’s motivation assumes, for all three methods: throughput scales from under 1 req/s at $c = 1$ to 21-42 req/s at $c = 100$ with clearly diminishing marginal returns above $c = 50$, TTFT p50 grows from 33-36 ms at $c = 1$ to 100-190 ms at $c = 100$, and mean KV-cache occupancy grows from under 0.25% to 3-17% depending on method, all consistent with a real PagedAttention-based serving engine under increasing concurrent load (Figure 6). The complete 35-cell aggregated table, including every method, ratio, and concurrency cell summarized above, is reproduced in Appendix G and archived as `results/sweep_summary.csv` (Section 10); the underlying per-request Locust and `/metrics` logs this table was computed from were generated on a rented cloud GPU instance and are not currently archived (see Data Availability), a gap the corrected v2 re-run (Section 8) is designed to close alongside the harness fixes.

5.9 Human Evaluation Results

Two bilingual evaluators independently rated all 90 blind (question, method) answer pairs specified in Section 4.10 (30 ARCD questions x raw/LSPM/naive truncation, $r = 0.3$). Table 9 reports mean correctness (0-2) and faithfulness (0-1) per method with 95% bootstrap confidence intervals.

Inter-rater agreement is substantial to near-perfect: unweighted Cohen’s kappa is 0.794 for correctness (three-level scale) and 0.935 for faithfulness (binary), computed on all 90 items across both raters.

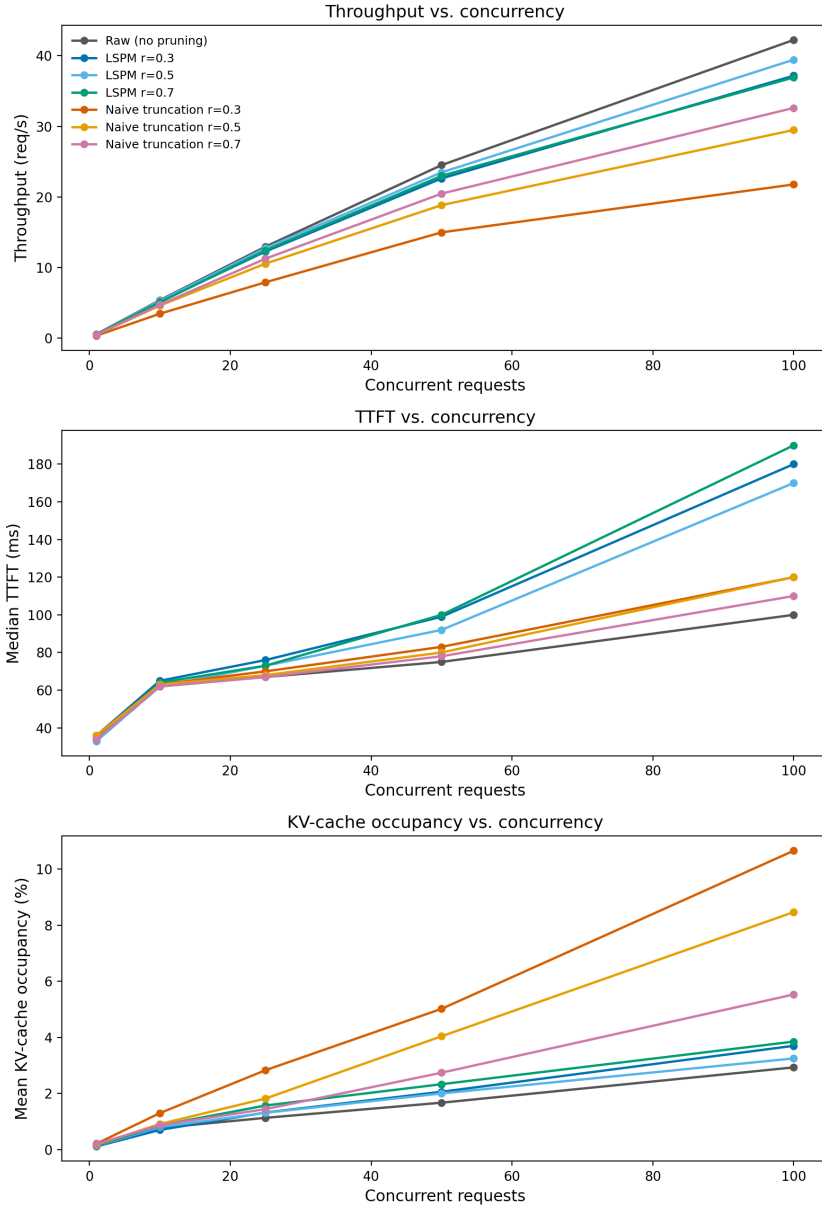


Fig. 6 three panels (throughput, TTFT, KV-cache occupancy) vs. concurrency, one line per method/ratio, from the measured 35-cell sweep. Read alongside the $c = 1$ -only claim above: the plotted curves make visually clear that the raw/LSPM/naive lines separate and reorder as concurrency increases, which is precisely the pattern this section reports as unresolved rather than confirmed

Table 9 Human-rated correctness and faithfulness by method (n = 30 items per method, two raters averaged per item, 95% bootstrap CI)

Method	n	Correct. (0-2)	Correct. CI	Faithful. (0-1)	Faithful. CI
Raw	30	1.717	[1.467, 1.917]	0.933	[0.833, 1.000]
LSPM (r=0.3)	30	1.800	[1.583, 1.950]	0.983	[0.950, 1.000]
Naive (r=0.3)	30	1.550	[1.250, 1.817]	0.800	[0.633, 0.933]

Human judgment agrees directionally with the automatic-metric result at this ratio (Section 5.4): LSPM’s mean correctness (1.800) is at least as high as raw’s (1.717) and its mean faithfulness (0.983) is at least as high as raw’s (0.933), with both point estimates for LSPM at or above raw’s on both dimensions; naive truncation scores lowest on both, most noticeably on faithfulness (0.800 vs. LSPM’s 0.983), consistent with a length-matched truncation more often cutting into content the answer’s supporting claims depend on. We report these as point estimates with overlapping confidence intervals at this sample size (n = 30 per method) rather than as a formally significant three-way comparison; no pairwise significance test between methods on the human-rated scores is claimed here, and the intervals in Table 9 should be read alongside the means rather than the means alone. What this result adds is qualitative: a blind, independent human read of the same 90 answers underlying part of Section 5.4’s automatic-metric comparison does not contradict that comparison, and directionally reinforces it, on a small, single-ratio, exploratory sample that closes the “automatic metrics only” gap flagged in Section 7 for this specific ratio and sample size, not for the full ratio range.

5.10 Edge-Cloud Small Validation Grid: Preliminary Results

This subsection reports the 90-cell grid described in Section 4.11: the fully corrected `kv_aware/network_aware` re-run (36 cells) alongside the unaffected `raw/naive/fixed_lspm` cells (54 cells) from the original run, plus, where informative, the pre-fix `kv_aware/network_aware` numbers for direct comparison. All 126 cells run across both the original and corrected sweeps (90 + 36) completed with zero request failures.

Both controllers now use a real signal instead of a hardcoded fallback, confirmed by a change in the observed ratio distribution. Before the fix, `kv_aware` selected exactly the fallback ratio (0.500) on every single request in both profiles at every concurrency level, the direct fingerprint of the metric-name bug (Section 4.11): its GPU-cache fetch always failed, and `get_ratio()` always took the “metrics unavailable” branch. After the fix, `kv_aware`’s fetch succeeds (zero fallback usage in the re-run) and it consistently selects 0.700, the ratio corresponding to low measured KV-cache pressure, which is correct behavior given this small manifest never drives sustained GPU KV-cache occupancy high enough to cross the controller’s higher-load thresholds (Section 5.10, KV-cache discussion below); this run confirms the fetch and mapping work end to end, not that the controller’s full dynamic range was exercised.

`network_aware` now differentiates its ratio by network profile; before the fix it did not. Before the fix, `network_aware` selected the same ratio in both profiles at a given concurrency (0.700 at $c = 1/10$, 0.555 at $c = 50$ in both `edge_lan` and `constrained_wireless`), the fingerprint of the real-loopback-probe bug: the controller was scoring the actual unshaped link, blind to which emulation profile a request was tagged with. After the fix, the controller’s continuous pressure score differs meaningfully by profile (mean 0.674 in `edge_lan` vs. 0.530 in `constrained_wireless`, $n = 5,443$ and 3,498 decisions respectively), and consistently snaps to a different validated ratio per profile: 0.700 in `edge_lan` (every decision) and 0.500 in `constrained_wireless` (every decision). This is the one result in this subsection we treat as a clean, positive confirmation: the network-aware controller responds to the network condition it is meant to respond to, in the correct direction (more compression under the more constrained profile), which it demonstrably did not do before the fix.

TTFT for the two adaptive controllers is substantially higher than for the three non-adaptive methods, and the fix changed, but did not remove, this gap. Table 10 reports mean TTFT (ms) per method, profile, and concurrency level (mean of each cell’s Locust-reported average TTFT across 3 repeats).

Table 10 Mean TTFT (ms) by method, profile, and concurrency (3-repeat mean per cell). C = concurrency; `constr_wireless` = `constrained_wireless`; `lspm` = `fixed_lspm`; `kv/net` = `kv_aware/network_aware`; (pre)/(post) = before/after the Section 4.11 bug fix

Profile	C	raw	naive	lspm	kv (pre)	kv (post)	net (pre)	net (post)
edge_lan	1	34	33	33	33	57	32	56
edge_lan	10	59	61	63	77	107	72	113
edge_lan	50	78	74	125	562	543	374	715
constr_wireless	1	94	92	93	96	122	94	116
constr_wireless	10	118	119	118	129	161	123	165
constr_wireless	50	137	135	165	680	553	368	703

At $c = 1$ and $c = 10$, the three non-adaptive methods track each other closely within each profile, as expected since they issue no extra network calls per request; `fixed_lspm` costs little beyond `raw/naive` at these concurrencies. Both adaptive methods sit noticeably above the non-adaptive group even at $c = 1$ (roughly 20-30 ms higher), the fixed per-request cost of one live HTTP round trip to vLLM’s `/metrics` endpoint before the compression ratio can be decided. At $c = 50$, the gap widens sharply for both adaptive methods relative to `fixed_lspm` (which itself is already elevated over `raw/naive` at $c = 50$, 125 ms vs. 74-78 ms in `edge_lan`, consistent with CPU-bound cross-encoder scoring under load, the same effect documented for the GPU benchmark in Section 5.8): `kv_aware` and `network_aware` both land in the 543-715 ms range after the fix, four to nine times `fixed_lspm`’s TTFT in the same cell.

Fixing the bugs made `kv_aware` somewhat faster at $c = 50$ but made `network_aware` substantially slower at every concurrency level tested, an unmasking rather than a regression. Before the fix, `kv_aware` issued two blocking synchronous HTTP calls per request (both wasted, since the metric-name bug meant

both always returned nothing usable); after the fix it issues exactly one, non-blocking, call. At $c = 50$ this nets out as a real improvement (562 to 543 ms in `edge_lan`, 680 to 553 ms in `constrained_wireless`). `network_aware`, by contrast, always issued exactly one such call, before and after the fix, so eliminating a redundant call cannot explain its change; instead, its TTFT rose at every concurrency level tested (roughly +24 ms at $c = 1$, +41 ms at $c = 10$, and +335 to +341 ms at $c = 50$ in both profiles). Our reading, consistent with the direction and the concurrency-scaling shape of the effect, is that the pre-fix code’s blocking synchronous call had an incidental side effect: it forced each request’s network/KV-cache probe to run one at a time on FastAPI’s single event loop, which, without our intending it, throttled how many concurrent hits vLLM’s `/metrics` endpoint actually received at once. The fix correctly makes this call properly concurrent, which is the change we set out to make, but that also means the endpoint now receives genuinely simultaneous polling from every in-flight request, and the resulting queueing cost is what Table 10 is measuring at $c = 50$. We did not further instrument the metrics endpoint itself to fully decompose this cost, so we report the pattern and our best-supported reading of it rather than a fully isolated causal claim. Either way, this is a real architectural property of the current design, one live per-request poll of `/metrics` per adaptive-controller call, not a residual bug; Section 4.11’s regression tests confirm the fetch, parsing, and profile-awareness are all now correct, and this scaling cost is what remains once those defects are removed. We treat it as a disclosed limitation (Section 7) with a specific proposed fix (periodic background polling instead of a live per-request fetch, Section 8) rather than something to paper over or silently re-tune away.

A KV-cache-occupancy and throughput comparison across raw/naive/fixed_lspm is directionally consistent with, but far too small a signal in this grid to confirm or extend, the Section 5.8 finding. GPU KV-cache occupancy in this grid never exceeds roughly 0.06% at any concurrency level tested (the 5-query mock manifest and single-turn requests used here are far smaller than what would meaningfully load a 24 GB GPU’s KV cache), so absolute differences between methods are close to measurement noise. At $c = 50$, `fixed_lspm`’s request throughput is 24.9 req/s in `edge_lan` (raw: 26.7, naive: 21.5) and 23.9 req/s in `constrained_wireless` (raw: 25.2, naive: 20.5), a modest, LSPM-favorable gap over naive truncation that is directionally consistent with Section 5.8’s throughput comparison, but we explicitly do not claim this replicates, extends, or resolves Section 5.8’s own reported KV-cache reversal at higher concurrency; that would require a manifest and concurrency range designed to actually stress the KV cache, which this grid, built to validate the edge-cloud plumbing and the two controllers rather than to re-run the GPU benchmark, was not.

6 Discussion

The tokenization-disparity measurement (5.1) establishes, with statistical confidence on the corpus tested, why Arabic RAG deployments face a KV-cache pressure problem that English deployments of the same architecture do not face to the same degree.

The ratio-swept fidelity pilot (5.2) shows fidelity remains high across a range of compression ratios on the original small corpus. The analytical projection (5.5) translates the real, measured context reduction into a concrete systems-relevant unit (MiB of KV-cache per request), and the measured GPU benchmark (5.8) provides one real, single-cell exploratory observation in the same direction on KV-cache: at $c = 1$, LSPM measurably reduces KV-cache occupancy against a live vLLM server, alongside an 8.5-11.9% lower request throughput that we report rather than round away, and a near-unchanged completion-token rate. We are explicit that this is a single concurrency level, not a low-concurrency range, since the KV-cache comparison is already mixed by $c = 10$ and reverses by $c \geq 25$, and that a harness fix and controlled re-run are needed before any claim beyond $c = 1$ can be trusted (5.8, 7, 8).

The blind human evaluation (5.9) speaks directly to a gap the automatic-metric results above cannot close on their own: whether a human reader, blind to method and unaware of the paper’s hypothesis, agrees that LSPM’s answers are correct and grounded, not merely lexically similar to a reference string. On the same 90 answers underlying part of the $r = 0.3$ ARCD comparison (5.4), two independent bilingual raters (substantial-to-near-perfect agreement, kappa 0.794/0.935) found LSPM’s mean correctness and faithfulness at least as high as the unpruned answer’s and higher than naive truncation’s, most visibly on faithfulness (0.983 vs. naive’s 0.800), which is consistent with a mechanism where content-aware pruning is less likely than blind length-matched truncation to cut a sentence a later claim in the answer actually depends on. We read this as directionally reinforcing, not independently proving at a formal significance threshold, the $r = 0.3$ Holm-Bonferroni result in 5.4: the sample is the same size ($n = 30$ per method) and the same single ratio, so it closes the “automatic metrics only” limitation specifically for $r = 0.3$, not across the full ratio range, and Section 8 identifies extending it to $r = 0.5/0.7$ as the natural next step.

The baseline comparison (5.3, re-tested at ARCD scale in 5.4) is the result that most shapes what this paper can and cannot claim, and we address it directly, including how our own reading of it has changed as we added statistical power. An earlier revision of this manuscript considered three, non-mutually-exclusive explanations for why LSPM’s cross-encoder scoring showed no significant head-to-head advantage over naive truncation on an 8-question, low-redundancy corpus: a corpus effect (curated passages where “first sentences” and “top-scored sentences” overlap heavily), limited statistical power at $n = 8$, and the possibility that semantic scoring’s advantage only manifests on longer, noisier, more redundant real-world retrieval results. A first ARCD re-test at $n = 40$ (five times the original pilot) left the head-to-head comparison still non-significant, which we reported honestly as unresolved rather than as a negative result, and flagged limited power as a live, not excluded, explanation: F1 favored LSPM by a consistent +0.078 to +0.090 across all three ratios at $n = 40$ without reaching significance, and a post-hoc power calculation on that effect size indicated roughly 115-140 questions per ratio would be needed for adequate power (previous revision, Section 8). We ran that fully powered re-test at $n = 140$ (Section 4.4/5.4), and it resolves the question at the most aggressive

ratio: at $r = 0.3$, LSPM significantly outperforms naive truncation head-to-head on F1 (+0.114, Holm-adjusted $p = 0.0054$), and is simultaneously shown statistically equivalent to the full unpruned answer by a predefined-margin equivalence test (TOST, $p = 0.038$), while naive truncation at the same ratio is significantly worse than the unpruned answer (adjusted $p = 0.00044$). This is the pattern the paper’s central hypothesis predicts: when compression is aggressive enough that content selection actually matters, cross-encoder relevance scoring keeps the answer where blind length-matched truncation does not. At $r = 0.5$ and $r = 0.7$, where naive truncation retains more of the pool regardless of position, the same directional advantage persists (+0.062, +0.058 F1) but does not survive correction (adjusted $p = 0.148$ at both), so the comparison remains genuinely open at those two ratios rather than resolved either way. One important scope caveat carries over unchanged from the $n = 40$ re-test: the gold, answer-bearing paragraph is included in every pool by construction (Section 4.4), so recall@6 is 100% by design, and this experiment evaluates sentence-level pruning conditional on the answer-bearing passage already having been retrieved, not end-to-end retrieval robustness.

We had previously flagged, and withdrawn, an invalid inference from the $n = 40$ data: reading LSPM’s non-significant difference from the unpruned answer alongside naive truncation’s nominally significant one as evidence of an LSPM-specific “fidelity preservation” effect. That inference remains invalid for the reason we gave at the time (the asymmetry is algebraically the same paired quantity as the head-to-head test, not independent evidence), and we do not resurrect it here. The $n = 140$ result reported above is a different, more defensible claim: it comes directly from a pre-specified, corrected head-to-head test crossing significance after Holm-Bonferroni adjustment, not from an asymmetry between two separate vs-raw comparisons. As a secondary, exploratory, non-inferential observation offered to help explain the mechanism rather than as a finding in itself: recomputing which sentence actually contains each gold answer string at $n = 140$ shows naive truncation retains that sentence in 74.3% of questions at $r = 0.3$, rising to 80.7% at $r = 0.5$ and 87.9% at $r = 0.7$ (i.e., naive truncation loses the answer sentence most often precisely at the ratio where the head-to-head effect is significant), while the six-document pool’s gold paragraph itself lands in the naive retriever’s first position in 98/140 (70.0%) of cases after keyword-overlap re-ranking, consistent with the $n = 40$ re-test’s 67.5%. This is a plausible mechanistic account of why the effect concentrates at $r = 0.3$ (naive truncation from the front of a re-ranked pool most often clips a genuinely different, non-first-ranked answer sentence exactly when the compression budget is tightest), and we present it as such, not as an independently tested claim.

On the systems side, StreamingLLM’s attention sinks [5] and H2O’s heavy-hitter eviction [6] both operate after tokens have already entered the KV cache; LSPM operates before the cache is populated at all, and at the most aggressive compression ratio tested, its content-aware selection now has statistically confirmed backing over the naive alternative for that role. Whether the same advantage holds at more moderate compression ratios remains an open, not yet statistically resolved, empirical question (Section 8).

The end-to-end retrieval check (4.9/5.7) asks a different, complementary question from the main re-test above: does the head-to-head advantage survive when retrieval itself can fail? With a real TF-IDF retriever pulling from a 234-paragraph corpus rather than a curated, answer-guaranteed pool, recall@6 dropped to 86.7%, and every method’s unconditional accuracy fell accordingly, exactly as expected when the correct text is genuinely absent for 13.3% of questions. Within that harder setting, LSPM’s advantage over naive truncation at $r = 0.3$ held in direction and rough magnitude (EM +0.100 overall, +0.096 on the retrieval-hit subset), and was nominally significant on EM before any correction ($p = 0.014$ overall, $p = 0.025$ on the hit subset), though we are explicit that this is a single-ratio, $n = 60$, uncorrected result, not a second instance of the Section 5.4 Holm-Bonferroni-confirmed finding. We read it as a reassuring, non-contradictory signal that the mechanism established under answer-guaranteed retrieval does not simply evaporate once retrieval is allowed to fail, not as independent statistical proof of the same magnitude. The retrieval-miss subset ($n = 8$) is too small to support inference, but is worth flagging descriptively: LSPM’s EM/F1 there (0.125/0.179) was not zero, unlike raw’s and naive’s (0.000/0.056 and 0.000/0.142), suggesting LSPM’s sentence-level relevance scoring may extract partial signal from a wrong-but-topically-adjacent context even when the true answer text is absent, a specific, testable hypothesis for future work rather than a claim we make here.

The preliminary edge-cloud extension (4.11/5.10) is a systems-integration check, not an accuracy result, and we read it accordingly. Its one clean positive finding is that the network-aware controller, once two real bugs were fixed, differentiates its compression-ratio decision by network condition (a mean continuous pressure score of 0.674 in the low-latency `edge_lan` profile vs. 0.530 in the constrained `constrained_wireless` profile, snapping to distinct validated ratios of 0.700 and 0.500 respectively), which it demonstrably failed to do beforehand. That the exercise also uncovered and fixed three real defects, a metric-name mismatch, blocking calls inside async handlers, and a controller blind to the active network-emulation profile, is itself informative in the same spirit as the client-side scoring bottleneck disclosed for the Section 5.8 GPU benchmark: preliminary systems work that has actually been run against live traffic surfaces integration bugs that a design document cannot. The less comfortable finding is that removing the blocking-call bug did not simply make the adaptive controllers faster; it made `network_aware`’s TTFT worse at every concurrency level tested (roughly +24 ms at $c = 1$ to +341 ms at $c = 50$), because the bug’s accidental serialization of `/metrics` polling had been masking a genuine cost of true concurrent polling that the fix correctly exposed rather than removed. We read this the same way we read the Section 5.8 reversal: as a real, diagnosed property of the current implementation to disclose and fix in a follow-up (Section 7, 8), not as a result to omit or explain away.

7 Limitations and Threats to Validity

Scale of the preliminary experiments. The tokenization-disparity corpus (30 sentence pairs) and the original fidelity/baseline pilot (8 questions $\times$ 3 ratios $\times$ 2

methods = 48 real live-endpoint data points) remain small by the standards of a mature empirical NLP evaluation; they were sized to be fully hand-verifiable rather than scraped or machine-generated at unlimited scale. The ARCD ground-truth re-test (140 questions $\times$ 7 cells = 980 real live-endpoint data points, Section 4.4/5.4) was deliberately sized larger, from a post-hoc power calculation on an earlier 40-question pilot, specifically to resolve the head-to-head comparison; it succeeds in doing so at $r = 0.3$ (Holm-adjusted $p = 0.0054$) but remains underpowered, or the true effect remains genuinely smaller, at $r = 0.5$ and $r = 0.7$ (adjusted $p = 0.148$ at both), so this specific, narrower question is only partially closed.

The GPU-scale systems result is real but limited to a single concurrency level, from a single run, comes with a real throughput tradeoff rather than a free win, and is affected by an identified methodological confound that is already visible at the very next concurrency level tested. Section 5.8 reports a measured, self-hosted vLLM benchmark (throughput, TTFT, KV-cache occupancy via Locust and the `/metrics` endpoint). Only at $c = 1$ does LSPM show lower KV-cache occupancy than raw at every tested ratio, and even there, request throughput is 8.5-11.9% lower than raw’s (completion tokens/second, a cleaner measure of model-side speed, is within 0.5%); at $c = 10$ the KV-cache comparison is already mixed (LSPM’s peak KV-cache exceeds raw’s at every ratio, and its mean exceeds raw’s at $r = 0.5$ and $r = 0.7$), and by $c \geq 25$ it reverses outright for both LSPM and naive truncation, in a way inconsistent with input-length reduction alone. We attribute this most plausibly to LSPM’s cross-encoder scoring running inside the same single-process Locust load generator (a client-side bottleneck under high simulated concurrency) and to all three method blocks running sequentially against one continuously running server without a restart between them, confounding elapsed run time with method identity. We do not claim a validated throughput or KV-cache advantage over raw or naive truncation at $c \geq 10$ from this run, we treat only the single $c = 1$ cell as a suggestive, exploratory observation, this was a single, non-replicated sweep rather than repeated runs with error bars, and the specific harness changes needed (precomputing pruned/truncated contexts outside the load-generator process; randomizing cell execution order across the full grid; repeated runs for confidence intervals) are given as required next steps in Section 8, not yet done.

The baseline comparison found a significant head-to-head LSPM advantage over naive truncation only at the most aggressive ratio tested ($r = 0.3$), after Holm-Bonferroni correction; at $r = 0.5$ and $r = 0.7$ the comparison remains statistically unresolved. On the original 8-question pilot (Section 5.3), all head-to-head p -values exceed 0.2. On the 140-question, ground-truth-scored ARCD re-test (Section 5.4), the $r = 0.3$ head-to-head comparison is significant after correction (adjusted $p = 0.0054$), and LSPM is additionally shown equivalent to the unpruned answer by TOST at $r = 0.3$ and $r = 0.7$. At $r = 0.5$ and $r = 0.7$ the head-to-head advantage remains directionally positive (+0.062, +0.058 F1) but does not survive correction (adjusted $p = 0.148$ at both), so we cannot rule out that a true advantage exists at those ratios but remains below this test’s detection threshold, nor can we claim one has been demonstrated there. Confirming or ruling out the head-to-head effect at $r = 0.5/0.7$ at still higher statistical power remains an open empirical

question, though a materially smaller and lower-priority one than before this revision’s re-test, and Section 8 discusses what further scale would be needed.

The $r = 0.3$ result rests on a single fully powered re-test, not an independent replication. We ran one properly powered ARCD re-test, not a second independent sample at the same scale; the usual caveats about a single significant result in a single dataset (this specific 140-question ARCD sample, this specific model, this specific retrieval pool construction) apply, and Section 8 identifies independent replication as a natural next step before treating the $r = 0.3$ finding as fully established.

The main ARCD re-test guarantees retrieval success by construction; the smaller end-to-end check (4.9/5.7) removes that guarantee but does not fully replace the main test’s statistical power. The gold, answer-bearing paragraph is included in every six-document candidate pool in Section 4.4, so recall@6 is 100% by design there; that experiment measures how LSPM and naive truncation prune a pool known to contain the answer, not how they behave under realistic retrieval failure. Section 4.9/5.7 addresses this directly with a real TF-IDF retriever over a 234-paragraph corpus (recall@6 = 86.7%, so retrieval genuinely fails on 8/60 questions), and finds the same directional LSPM-vs-naive advantage there. But this check is a single ratio ($r = 0.3$), $n = 60$, uncorrected-p-value result, not a second instance of the Section 5.4 finding at the same statistical standard; a fully powered, Holm-Bonferroni-corrected end-to-end evaluation, ideally against a production-scale dense or hybrid retriever over a much larger real corpus, remains future work (Section 8).

Single generation model. All fidelity and baseline results use one instruction-tuned model (Llama-3.1-8B-Instruct) accessed through a hosted endpoint. Results may not transfer identically to other model families or sizes.

Retriever simplicity. The reference implementation’s default retriever uses simple keyword overlap, and the ARCD re-test’s noisier six-document pool (Section 4.4) is still a small, fixed candidate set assembled from a benchmark. The end-to-end check (Section 4.9) upgrades this to a real TF-IDF retriever over a 234-paragraph corpus, but that corpus and retriever are still far smaller and simpler than a production-scale dense or hybrid retrieval stack searching a large real corpus. LSPM is designed to be retriever-agnostic, but evaluation with a genuinely production-scale retrieval stack in front of it remains to be done.

Automatic metrics as fidelity proxies – now partially, not fully, addressed by human evaluation. The original pilot’s BLEU, ROUGE-L, and BERTScore measure similarity to the unpruned answer, not correctness; the ARCD re-test (Section 4.4/5.4) addresses this specific gap by scoring against ARCD’s real gold answers with Exact Match and token-F1, and Section 4.10/5.9 adds a blind, two-rater human evaluation (correctness and faithfulness, 90 items, substantial-to-near-perfect inter-rater agreement) that directionally agrees with the automatic-metric result. This closes the gap specifically at $r = 0.3$, on a single 90-item sample, with point estimates rather than a separately powered significance test between methods on the human-rated scores; it has not been run at $r = 0.5$ or $r = 0.7$, nor at a sample size large enough to detect smaller between-method differences on the human scale itself. Exact Match in particular remains a strict, brittle automatic metric at this sample size. RAG-specific reference-free metrics such as RAGAS [37], and extending the human

evaluation across all three ratios with a larger sample, would strengthen the claim further in a follow-up study.

LSPM overhead measurement is CPU-only and single-request. Section 5.6’s 57.8 ms figure does not characterize batched or concurrent-load latency, nor GPU-accelerated cross-encoder inference, which a production vLLM deployment would likely use.

The edge-cloud validation grid (4.11/5.10) is small-scale, single-box, application-emulated, and carries no correctness data. All 90 cells used a 5-query mock corpus, not an ARCD-scale evaluation, so nothing in Section 5.10 speaks to answer accuracy or faithfulness; those remain governed entirely by Sections 5.4, 5.7, and 5.9. The edge gateway and cloud vLLM server run on the same physical GPU instance, so no result here reflects a genuinely distributed deployment or a real wide-area network link. Network conditions were emulated at the application level (per-request sleeps and probabilistic drops inside the gateway), not with kernel-level `tc netem`, because the rented container did not grant `CAP_NET_ADMIN`; this is a coarser model of loss (whole-request rather than per-packet) and has no TCP-level interaction, so it should not be read as equivalent to a kernel-shaped link. The two rule-based controllers’ calibration ranges and weights are explicitly placeholder values, not fitted to measured data, so the grid validates that the controllers respond to real signals in the right direction, not that their specific thresholds are well tuned. And, disclosed at length in Section 5.10 and Section 6, the current design’s one-/metrics-poll-per-request pattern is a genuine, now-unmasked scaling cost under concurrency for both adaptive controllers, most visibly `network_aware` at $c = 50$, which the next revision needs to address rather than re-hide behind a different bug.

8 Future Work

The fully powered head-to-head re-test called for in the previous revision has now been run (Section 4.4/5.4): a post-hoc power calculation on the $n = 40$ pilot’s observed F1 differences (Cohen’s $d \approx 0.24$ - 0.26 across ratios) indicated approximately 115-140 paired ARCD questions per ratio would be needed for 80% power, and we ran the identical ARCD pipeline at $n = 140$. This resolved the question at $r = 0.3$ (Holm-adjusted $p = 0.0054$, LSPM significantly better, and equivalent to unpruned by TOST) but left $r = 0.5$ and $r = 0.7$ still non-significant after correction (adjusted $p = 0.148$ at both). Two items are now the top remaining priorities. **First**, extending the power analysis to the two still-unresolved ratios specifically: the observed $r = 0.5/0.7$ effect sizes ($+0.062$, $+0.058$ F1) are smaller than the $r = 0.3$ effect that motivated the $n = 140$ target, so resolving them with 80% power would require a further sample-size increase beyond 140, which a follow-up study should compute directly from this revision’s effect sizes rather than reusing the $r = 0.3$ -derived target; independent replication of the $r = 0.3$ result on a fresh ARCD sample (or a second QA benchmark) would also strengthen that finding beyond a single fully powered run. **Second**, the GPU-scale throughput and KV-cache benchmark specified in Section 4.6 has now been run once (Section 5.8): a self-hosted vLLM server on a single RTX 3090, the full $\{0.3, 0.5, 0.7\}$ ratio $\times \{1, 10, 25, 50, 100\}$ concurrency matrix, for raw, LSPM, and naive

truncation (35 cells, 90 s each). Only the single $c = 1$ cell supports LSPM’s measured KV-cache reduction as an exploratory observation, and even there it comes with an 8.5-11.9% lower request throughput alongside a near-unchanged completion-token rate, not a cost-free result; the KV-cache comparison is already mixed at $c = 10$ and confounded from $c \geq 25$ on, most likely by LSPM’s cross-encoder scoring running inside the same single-process Locust load generator that issues requests, and by all three method blocks running sequentially against one continuously running server without a restart between them. This is the top remaining priority for the next revision, not a lower-priority polish item: two specific harness fixes are needed before any concurrency level beyond $c = 1$ can be trusted, precomputing pruned and truncated contexts outside the load-generator process (so pruning cannot throttle request submission under Locust’s cooperative concurrency at any simulated user count, removing the confound for all three methods, not just LSPM), and randomizing cell execution order across the full 35-cell grid (rather than running fixed method blocks in sequence) so elapsed run time is no longer confounded with method identity, together with repeated runs at each cell to report confidence intervals rather than a single sweep. A follow-up run with all three fixes applied, and a corresponding new tagged release and Zenodo version archiving that run’s full raw per-request output (closing the specific archival gap disclosed in the Data Availability statement for the current run), would let the full concurrency range be reported without qualification.

A first end-to-end retrieval check, with recall no longer guaranteed, has also now been run (Section 4.9/5.7): a real TF-IDF retriever over a 234-paragraph corpus, $\text{recall@6} = 86.7\%$, 60 questions at $r = 0.3$ only. It found the same directional LSPM-vs-naive advantage (EM +0.100, uncorrected $p = 0.014$), which is reassuring but explicitly not a fully powered or corrected confirmation. The natural next step is to scale this exact check up (a larger corpus, a production-scale dense or hybrid retriever in place of TF-IDF, and a sample size derived from this check’s own observed effect size) to the same Holm-Bonferroni-corrected standard as Section 5.4, and to extend it across all three compression ratios rather than $r = 0.3$ alone.

A small, blind, two-rater human evaluation, previously listed as future work, has also now been run (Section 4.10/5.9): 90 answers (30 ARCD questions $\times$ raw/LSPM/naive truncation, $r = 0.3$), rated for correctness and faithfulness with substantial-to-near-perfect inter-rater agreement (κ 0.794, 0.935), directionally agreeing with the automatic-metric result at that ratio. The natural next step is the same one that applies to the accuracy re-test itself: extend the human evaluation across $r = 0.5$ and $r = 0.7$, and to a larger sample sized from this check’s own observed between-method differences, so that human judgment can speak to the same full ratio range the automatic metrics already cover, and add a formal significance test between methods on the human-rated scores rather than reporting point estimates and confidence intervals alone.

The preliminary edge-cloud extension (4.11/5.10) has now been run once, corrected once, and re-run once; the top remaining priority for it is fixing the per-request /metrics-polling cost it surfaced, not re-running the same grid at larger scale. A periodic background poll of vLLM’s /metrics endpoint (a single shared, refreshed-on-an-interval KV-cache-usage value read by all

in-flight requests, rather than one live HTTP round trip per request) is the specific fix the Section 5.10 finding points to, and should be re-validated on the same 36 `kv_aware/network_aware` cells before any larger sweep is attempted. Beyond that fix: running the full four-profile spectrum defined in `network_profiles.py` (this grid used only the two most separated profiles) and the same $\{1, 10, 50\}$ or a wider concurrency range at a scale comparable to Section 4.6’s 35-cell sweep; fitting the two controllers’ placeholder calibration ranges and weights to a measured bandwidth/RTT/queue-length/KV-cache distribution instead of the current named-but-unfitted constants; obtaining kernel-level `tc netem` access (a container with `CAP_NET_ADMIN`, or a non-containerized host) to replace the application-level emulation with per-packet shaping and remove the whole-request-loss and no-TCP-interaction simplifications disclosed in Section 7; running the edge gateway and cloud server on physically separate machines rather than co-located on one GPU instance; and, only once the above are in place, extending the grid to the ARCD-scale corpus and adding an answer-correctness/faithfulness pass (mirroring Section 4.10’s human evaluation) so the edge-cloud extension can eventually be evaluated on the same accuracy standard as the rest of this paper, not only on latency and ratio-decision correctness.

Additional future work: incorporating RAGAS-style reference-free faithfulness and context-relevance metrics [37] alongside BLEU/ROUGE-L/BERTScore/EM/F1; quantifying LSPM’s overhead under batched/concurrent load rather than the single-request CPU measurement in Section 5.6; quantifying the compositional benefit of combining LSPM with KV-cache-side management techniques such as StreamingLLM [5] or H2O [6]; testing the specific, currently-descriptive-only observation that LSPM retains partial signal when retrieval fails outright (Section 5.7’s $n = 8$ retrieval-miss subset) with an adequately powered, dedicated sample; and extending the dynamic compression-ratio controller (Section 3.4), currently an untested design proposal rather than a validated result, beyond linear interpolation to a learned policy evaluated under realistic, bursty production traffic. One implication of the $r = 0.3$ -concentrated effect worth testing directly: since the dynamic controller is designed to tighten the compression ratio under memory pressure, a follow-up study could test whether it, in effect, automatically steers deployments toward the ratio regime (aggressive compression) where LSPM’s advantage over naive truncation is now confirmed.

9 Conclusion

Arabic RAG systems deployed on memory-efficient serving engines such as vLLM face a KV-cache pressure problem rooted in tokenization, which we confirm with statistical significance on the corpus tested, across two independent tokenizers (aggregate ratio 1.69x-1.76x, bootstrap $p < 0.0001$, $n = 30$). We introduced LSPM, a lightweight, sentence-level, cross-encoder-driven pruning middleware, and evaluated it with the same rigor and the same honesty regardless of outcome: a ratio-swept fidelity pilot ($r = 0.3$ - 0.7) shows preserved answer fidelity across the tested range on that pilot’s small corpus, and a paired baseline comparison against naive length-matched truncation, run first on a small curated pilot ($n = 8$) and then re-tested on 140 real, ground-truth-scored ARCD questions (980 live generations) against a noisier, answer-guaranteed

six-document pool, finds a statistically significant head-to-head advantage for LSPM’s semantic scoring at the most aggressive compression ratio tested ($r = 0.3$: $+0.114$ F1, Holm-Bonferroni-adjusted $p = 0.0054$), where LSPM is additionally shown statistically equivalent to the unpruned answer (TOST, $p = 0.038$) while naive truncation is significantly worse than the unpruned answer (adjusted $p = 0.0004$). At the more moderate ratios ($r = 0.5, 0.7$) the same directional advantage does not survive correction, so the comparison remains genuinely open there. A smaller, complementary check that replaces the answer-guaranteed retrieval pool with a real TF-IDF retriever over a larger corpus (recall@6 = 86.7%, so retrieval genuinely fails on some questions) finds the same directional advantage at $r = 0.3$, reported honestly as a single-ratio, uncorrected, exploratory result rather than a second fully powered confirmation. We report this exact pattern, resolved with statistical confidence at one ratio, open at two others, and directionally reinforced under harder, more realistic retrieval, rather than rounding it up to a uniform win or down to a uniform null, having caught and corrected our own invalid overreading of an earlier, smaller-sample asymmetry during a prior revision. We provide an analytical, explicitly-labeled projection connecting the real measured context reduction to KV-cache bytes saved (9.7-19.1 MiB per request depending on ratio), and we ran the GPU-scale benchmark this projection motivated: on a self-hosted vLLM server, LSPM’s measured KV-cache reduction holds as a single-cell, exploratory observation at $c = 1$, alongside a real 8.5-11.9% lower request throughput at that same cell that we report rather than describe as free, already weakens on the KV-cache dimension by $c = 10$, and reverses from $c \geq 25$ on, a pattern we diagnose rather than paper over, with the corrected, controlled re-run specified as the top remaining priority (Section 5.8, 8). A blind, two-rater human evaluation on 90 of the $r = 0.3$ answers (Section 5.9), with substantial-to-near-perfect inter-rater agreement (kappa 0.794 correctness, 0.935 faithfulness), directionally agrees with the automatic-metric result at that ratio, closing the “automatic metrics only” gap for this specific ratio and sample size. As a separate, preliminary extension in a different direction, we also implemented and ran a small edge-cloud validation grid (90 cells, Section 4.11/5.10) with two new rule-based, non-learned compression-ratio controllers; the exercise found and fixed three real implementation defects, and, once fixed, showed the network-aware controller correctly differentiating its ratio decision by network condition, alongside a genuine, now-disclosed architectural cost from per-request metrics polling under concurrency that the fix unmasked rather than removed. This extension is small-scale, single-box, and carries no correctness or faithfulness data, and we report it as exactly that, a systems-integration check with one clean positive result and one clearly diagnosed limitation, not as grounds to retitle or reframe this paper around network-awareness. We specify, in full, both that remaining benchmark work and the ratio-specific higher-power re-test still needed, for both the automatic and the human-rated comparisons, to convert this paper’s architecture and preliminary evidence into a fully validated systems result. We release the complete implementation, all raw data from every round of experiments including both ARCD re-tests and the end-to-end retrieval check, and a live demonstration interface to support independent reproduction and extension of every claim in this paper, including the significant, null, and inconclusive results alike.

Statements and Declarations

Data availability. The complete source code (LSPM middleware, evaluation scripts, statistical analysis scripts, benchmarking harness, and demonstration interface) is publicly available at <https://github.com/Akramtaha98/vllm-arabic-rag>. The 30-pair Arabic-English tokenization corpus, the 8-question/10-document fidelity and baseline pilot dataset (all 48 ratio×method cells across its single- and multi-ratio phases), the ARCD ground-truth re-test dataset and code (`build_arcd_eval_set.py`, `arabic_em_f1.py`, `run_arcd_pilot.py`, `analyze_arcd_results.py`, and the full 980-row `arcd_results.jsonl` output at $n = 140$, which contains the original $n = 40$ sample as its first 280 rows), the end-to-end retrieval check dataset and code (`build_e2e_retrieval_eval.py`, `run_e2e_retrieval_pilot.py`, `analyze_e2e_retrieval_results.py`, and the 180-row `e2e_retrieval_results.jsonl` output), and all raw experimental outputs for these three experiment families (tokenization CSVs, raw/pruned/naive answer triples, fidelity metric scores, bootstrap analysis outputs, the Holm-Bonferroni and TOST statistical outputs, and the analytical KV-cache estimate script and its output) are publicly available in the same repository. **For the Section 5.8 GPU benchmark specifically:** the aggregated 35-cell summary (`results/sweep_summary.csv`) and comparison figure (`results/sweep_comparison.png`, Figure 6) are pushed to the repository; the underlying per-request Locust and vLLM `/metrics` logs (140 files) were generated on a temporary rented cloud GPU instance and were not transferred off it before this submission, so they are not currently archived. This is disclosed here rather than implied to be complete, and closing it is tracked as a required step (Section 8) alongside the corrected v2 re-run, whose full raw output will be archived when it is run. **For the Section 4.10/5.9 human evaluation:** the rubric (`HUMAN_EVAL_PROTOCOL.md`), the completed blind rating sheet with both raters' scores (`data/human_eval_sample_blind.csv`), and the summary statistics (`results/human_eval_summary.csv`) are included in the repository; the private item-to-method codebook (`data/human_eval_codebook.json`), used only to keep raters blind during rating and no longer needed once rating was complete, is released alongside them. The repository state described above is tagged release `v1.0-submission`, archived with DOI [10.5281/zenodo.21826992](https://doi.org/10.5281/zenodo.21826992) [43]; a version-specific DOI update following the v2 re-run and a corresponding new Zenodo version is anticipated before this manuscript is finalized. **For the Section 4.11/5.10 edge-cloud validation grid:** the edge gateway, network-emulation, and controller source (`benchmark/edge_gateway.py`, `benchmark/network_profiles.py`, `benchmark/network_controller.py`, `benchmark/locustfile_edge.py`, `benchmark/run_validation_grid.py`, `benchmark/request_manifest.py/request_manifest.json`) and the corresponding additions to `middleware/pruning.py` are pushed to the repository. Raw output from both runs is included: the original 90-cell run (`results/validation_grid/`, commit 3972447, containing the still-valid `raw/naive/fixed_lspm` cells and the since-corrected pre-fix `kv_aware/network_aware` cells), the bug-fix commit (d04ff12), and the corrected 36-cell re-run (`results/validation_grid_rerun2/`, commit 7cb9b7e), each containing per-cell Locust stats/history/failures/exceptions CSVs

and vLLM `/metrics` polling CSVs, plus the cumulative per-request edge-gateway log (`results/edge_gateway_log.jsonl`, approximately 83 MB, spanning both runs) and the grid orchestrator’s own run log (`grid_run_log.jsonl` in each results directory). This data has not yet been tagged in a dedicated release or archived with a new Zenodo version; doing so alongside the next GPU-benchmark re-run (Section 8) is tracked as an open item, disclosed here rather than implied to be complete.

Ethics declaration. This study did not involve human participants as research subjects, personal data, or clinical data. The parallel-corpus and pilot-corpus text was authored by the researcher for evaluation purposes and describes only publicly available, non-sensitive institutional information. Section 4.10/5.9’s human evaluation used two volunteer bilingual annotators who rated model outputs on a rubric (a standard NLP quality-control annotation task, not an intervention on or study of the annotators themselves); no personal, demographic, or identifying data about the annotators was collected or is reported, and their ratings are reported only in aggregate. No ethics committee approval was sought or required for this annotation task, consistent with standard practice for NLP evaluation studies of this kind.

Author contributions (CRediT). Akram Taha: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Conflict of interest. The author declares no conflict of interest.

Funding. This research received no external funding. Inference costs for the preliminary experiments were incurred against a free-tier hosted API allowance.

AI usage disclosure. See Section 4.8 for the full disclosure of AI assistance used in this project.

References

- [1] A. Vaswani et al., "Attention Is All You Need," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017, doi: [10.48550/arXiv.1706.03762](https://doi.org/10.48550/arXiv.1706.03762)
- [2] T. B. Brown et al., "Language Models are Few-Shot Learners," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020, doi: [10.48550/arXiv.2005.14165](https://doi.org/10.48550/arXiv.2005.14165)
- [3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020, doi: [10.48550/arXiv.2005.11401](https://doi.org/10.48550/arXiv.2005.11401)
- [4] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proc. 29th ACM Symp. Operating Systems Principles (SOSP), 2023, doi: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165)
- [5] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," in Proc. Int. Conf. Learning Representations (ICLR), 2024, doi: [10.48550/arXiv.2309.17453](https://doi.org/10.48550/arXiv.2309.17453)

- [6] Z. Zhang et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2023, doi: [10.52202/075280-1506](https://doi.org/10.52202/075280-1506)
- [7] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," in Proc. 16th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2022.
- [8] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL), 2016, pp. 1715–1725, doi: [10.18653/v1/P16-1162](https://doi.org/10.18653/v1/P16-1162)
- [9] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023, doi: [10.18653/v1/2023.emnlp-main.825](https://doi.org/10.18653/v1/2023.emnlp-main.825)
- [10] Y. Li, B. Dong, C. Lin, and F. Guerin, "Compressing Context to Enhance Inference Efficiency of Large Language Models," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023, doi: [10.18653/v1/2023.emnlp-main.391](https://doi.org/10.18653/v1/2023.emnlp-main.391)
- [11] F. Xu, W. Shi, and E. Choi, "RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation," in Proc. Int. Conf. Learning Representations (ICLR), 2024, doi: [10.48550/arXiv.2310.04408](https://doi.org/10.48550/arXiv.2310.04408)
- [12] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in Proc. 43rd Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 2020, doi: [10.1145/3397271.3401075](https://doi.org/10.1145/3397271.3401075)
- [13] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv preprint arXiv:1901.04085, 2019, doi: [10.48550/arXiv.1901.04085](https://doi.org/10.48550/arXiv.1901.04085)
- [14] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023, doi: [10.48550/arXiv.2312.10997](https://doi.org/10.48550/arXiv.2312.10997)
- [15] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in Proc. Int. Conf. Learning Representations (ICLR), 2024, doi: [10.48550/arXiv.2310.11511](https://doi.org/10.48550/arXiv.2310.11511)
- [16] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009, doi: [10.1561/15000000019](https://doi.org/10.1561/15000000019)

- [17] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, doi: [10.18653/v1/2020.emnlp-main.550](https://doi.org/10.18653/v1/2020.emnlp-main.550)
- [18] N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157–173, 2024, doi: [10.1162/tacl.a_00638](https://doi.org/10.1162/tacl.a_00638)
- [19] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," in Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), 2019, doi: [10.18653/v1/D19-1410](https://doi.org/10.18653/v1/D19-1410)
- [20] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 Text Embeddings: A Technical Report," arXiv preprint arXiv:2402.05672, 2024, doi: [10.48550/arXiv.2402.05672](https://doi.org/10.48550/arXiv.2402.05672)
- [21] X. Zhang et al., "MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages," Transactions of the Association for Computational Linguistics, vol. 11, pp. 1114–1131, 2023, doi: [10.1162/tacl.a_00595](https://doi.org/10.1162/tacl.a_00595)
- [22] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," in Proc. 17th Conf. European Chapter of the Association for Computational Linguistics (EACL), 2023, doi: [10.18653/v1/2023.eacl-main.148](https://doi.org/10.18653/v1/2023.eacl-main.148)
- [23] W. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding," in Proc. 4th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT), 2020, doi: [10.48550/arXiv.2003.00104](https://doi.org/10.48550/arXiv.2003.00104)
- [24] W. Antoun, F. Baly, and H. Hajj, "AraGPT2: Pre-Trained Transformer for Arabic Language Generation," in Proc. 6th Arabic Natural Language Processing Workshop (WANLP), 2021, doi: [10.48550/arXiv.2012.15520](https://doi.org/10.48550/arXiv.2012.15520)
- [25] H. Mozannar, K. El Hajal, E. Maamary, and H. Hajj, "Neural Arabic Question Answering," in Proc. 4th Arabic Natural Language Processing Workshop (WANLP), 2019, doi: [10.18653/v1/W19-4612](https://doi.org/10.18653/v1/W19-4612)
- [26] N. Sengupta et al., "Jais and Jais-chat: Arabic-Centric Foundation and Instruction-Tuned Open Generative Large Language Models," arXiv preprint arXiv:2308.16149, 2023, doi: [10.48550/arXiv.2308.16149](https://doi.org/10.48550/arXiv.2308.16149)
- [27] A. Huang et al., "ACVA: Arabic Cultural and Value Alignment Benchmark," Hugging Face Dataset, 2024. [Online]. Available: <https://huggingface.co/datasets/FreedomIntelligence/ACVA-Arabic-Cultural-Value-Alignment>

- [28] F. Koto et al., "ArabicMMLU: Assessing Massive Multitask Language Understanding in Arabic," 2024, doi: [10.18653/v1/2024.findings-acl.334](https://doi.org/10.18653/v1/2024.findings-acl.334)
- [29] O. Obeid et al., "CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing," in Proc. 12th Language Resources and Evaluation Conf. (LREC), 2020, pp. 7022–7032, doi: [10.63317/295vncriwcn2](https://doi.org/10.63317/295vncriwcn2)
- [30] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2022, doi: [10.52202/068431-1189](https://doi.org/10.52202/068431-1189)
- [31] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in Proc. Int. Conf. Learning Representations (ICLR), 2023, doi: [10.48550/arXiv.2210.17323](https://doi.org/10.48550/arXiv.2210.17323)
- [32] J. Lin et al., "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration," in Proc. Machine Learning and Systems (MLSys), 2024, doi: [10.48550/arXiv.2306.00978](https://doi.org/10.48550/arXiv.2306.00978)
- [33] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in Proc. 40th Int. Conf. Machine Learning (ICML), 2023, doi: [10.48550/arXiv.2211.17192](https://doi.org/10.48550/arXiv.2211.17192)
- [34] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL), 2002, pp. 311–318, doi: [10.3115/1073083.1073135](https://doi.org/10.3115/1073083.1073135)
- [35] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in Text Summarization Branches Out: Proc. ACL Workshop, 2004, pp. 74–81.
- [36] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in Proc. Int. Conf. Learning Representations (ICLR), 2020, doi: [10.48550/arXiv.1904.09675](https://doi.org/10.48550/arXiv.1904.09675)
- [37] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL), 2024, doi: [10.18653/v1/2024.eacl-demo.16](https://doi.org/10.18653/v1/2024.eacl-demo.16)
- [38] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall/CRC, 1993, doi: [10.1201/9780429246593](https://doi.org/10.1201/9780429246593)
- [39] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021, doi: [10.1109/TBDDATA.2019.2921572](https://doi.org/10.1109/TBDDATA.2019.2921572)

- [40] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv preprint arXiv:2402.03216, 2024, doi: [10.48550/arXiv.2402.03216](https://doi.org/10.48550/arXiv.2402.03216)
- [41] Qwen Team, "Qwen2.5 Technical Report," arXiv preprint arXiv:2412.15115, 2024, doi: [10.48550/arXiv.2412.15115](https://doi.org/10.48550/arXiv.2412.15115)
- [42] AI@Meta, "The Llama 3 Herd of Models," arXiv preprint arXiv:2407.21783, 2024, doi: [10.48550/arXiv.2407.21783](https://doi.org/10.48550/arXiv.2407.21783)
- [43] A. Taha, "Semantic-Driven Context Pruning for Arabic RAG Systems: Toward Memory-Efficient vLLM-Based Deployment — Code and Data," Version v1, Zenodo, 2026, doi: [10.5281/zenodo.21826992](https://doi.org/10.5281/zenodo.21826992)